Apostila 2 — pandas para Economia (Teoria Completa)

Teoria, exemplos executados com dados reais e armadilhas, com foco em economia

Sumário

Apostila 2 — pandas para Economia (Teoria Completa) 1
Como Usar Esta Apostila 4
Capítulo 1 — O que é pandas e por que ele existe 4
1.1. Um pouco de história 4
1.2. pandas é construído sobre numpy — e isso importa 5
1.3. Os dois objetos centrais: Series e DataFrame 5
1.4. O Index: o rótulo que a lista Python não tem 6
1.5. Instalação e convenção de import 7
1.6. Exemplo Resolvido — Alinhamento de Index em séries de fontes diferentes 7
Capítulo 2 — Criando DataFrames e lendo dados reais 8
2.1. A partir de estruturas Python 8
2.2. `read_csv` em detalhe — a função mais usada de pandas 9
2.3. `read_excel` 10
2.4. Fontes de dados econômicos brasileiros 10
2.5. Outras origens 10
2.6. Exemplo Resolvido — Importando um CSV "sujo" do jeito certo 11
Capítulo 3 — Explorando e entendendo a estrutura dos dados 13
3.1. Primeiro contato com uma tabela nova 13
3.2. `describe()`, `unique()`, `value_counts()` 14
3.3. Tipos de dados (dtypes) do pandas — mais do que parece 14
3.4. Exemplo Resolvido — Diagnosticando um DataFrame recém-importado 15
Capítulo 4 — Seleção de dados: `[]`, `.loc`, `.iloc`, `.at`, `.iat` 17
4.1. Selecionando colunas 17
4.2. `.iloc` — seleção por posição (i de "integer location") 17
4.3. `.loc` — seleção por rótulo ("location" pelo Index) 17
4.4. A diferença crucial entre `.loc` e `.iloc`: inclusivo vs. exclusivo 17
4.5. `.at` e `.iat` — acesso a um único valor, mais rápido 18
4.6. Exemplo Resolvido — Escolhendo a ferramenta certa de seleção 18
Capítulo 5 — Cópia vs. view: o `SettingWithCopyWarning` 20
5.1. Por que este capítulo existe 20
5.2. Reproduzindo o problema 20
5.3. As duas formas corretas de resolver 20
5.4. Tabela-resumo do capítulo 21
5.5. Exemplo Resolvido — Corrigindo um pipeline com aviso silencioso 21
Capítulo 6 — Filtragem booleana 22
6.1. O que realmente acontece por baixo dos panos 22
6.2. Combinando condições: por que `and`/`or` não funcionam 23
6.3. Métodos de filtro especializados 23
6.4. `.query()` — uma sintaxe alternativa mais legível 24
6.5. Exemplo Resolvido — Diagnosticando um filtro que retorna vazio 24
Capítulo 7 — Criando, transformando e removendo colunas 25
7.1. Atribuição direta vetorizada (a forma preferida) 25
7.2. `.apply()` com função ou `lambda` 25
7.3. Por que `.apply()` é mais lento — e quando isso importa 26
7.4. `.map()`, `.replace()` e renomeação/remoção de colunas 27
7.5. Exemplo Resolvido — Reescrevendo um `.apply()` como vetorizado 27
Capítulo 8 — Dados faltantes (`NaN`) em profundidade 28
8.1. O que é `NaN`, tecnicamente 28

8.2. Detectando dados faltantes 29
8.3. Removendo linhas/colunas com dado faltante 30
8.4. Preenchendo dados faltantes 30
8.5. Exemplo Resolvido — Diagnosticando a causa de um `NaN` antes de tratá-lo 30
Capítulo 9 — Agrupamento (`groupby`) 32
9.1. O paradigma split-apply-combine 32
9.2. Múltiplas agregações com `.agg()` 33
9.3. Agrupando por múltiplas colunas 33
9.4. `.transform()` vs. `.agg()` — resultado do mesmo tamanho ou reduzido 33
9.5. Exemplo Resolvido — Volatilidade do PIB por país, ordenada 34
Capítulo 10 — Combinando tabelas: `merge`, `concat`, `join` 35
10.1. `merge` — juntando por uma chave comum 35
10.2. `concat` — empilhando tabelas 36
10.3. `join` — atalho baseado no Index 37
10.4. Exemplo Resolvido — Cruzando três fontes com chaves diferentes 37
Capítulo 11 — Datas e séries temporais 39
11.1. `to_datetime` e o acessor `.dt` 39
11.2. Data como Index — a base de toda série temporal em pandas 39
11.3. `resample` — mudando a frequência de uma série temporal 39
11.4. Defasagem (lag), diferenças e variação percentual 40
11.5. Médias móveis (`rolling`) e janelas expansivas (`expanding`) 40
11.6. Exemplo Resolvido — IPCA acumulado em 12 meses, calculado do zero 40
Capítulo 12 — Estatística descritiva com pandas 42
12.1. Tendência central e dispersão 42
12.2. Correlação 43
12.3. Exemplo Resolvido — Comparando volatilidade entre duas décadas 43
Capítulo 13 — Exportando dados e um script completo do mundo real 45
13.1. Salvando resultados 45
13.2. Script completo — o que um economista faz no dia a dia 45
Capítulo 14 — Boas práticas com pandas 46
Capítulo 15 — Exercícios para Executar (na mão) 48
Exercício 1 — Trace de `.loc` vs. `.iloc` 48
Exercício 2 — Calculando correlação de Pearson na mão 48
Exercício 3 — Encontre o erro: `SettingWithCopyWarning` 48
Exercício 4 — Pseudocódigo do split-apply-combine 49
Exercício 5 — Prevendo dtypes após um merge externo 49
Exercício 6 — `rolling` vs. `expanding`, na mão 49
Capítulo 16 — Exercícios para Executar (em código) 49
Exercício 1 — Primeira exploração 49
Exercício 2 — `.loc` vs `.iloc` na prática 49
Exercício 3 — Reproduzindo o `SettingWithCopyWarning` 49
Exercício 4 — Filtro composto 49
Exercício 5 — `np.select` substituindo `.apply()` 49
Exercício 6 — Diagnóstico e tratamento de nulos 50
Exercício 7 — `groupby` com múltiplas agregações nomeadas 50
Exercício 8 — `transform` para desvio em relação ao grupo 50
Exercício 9 — Merge com `indicator` e auditoria 50
Exercício 10 — `validate` capturando um merge indevido 50
Exercício 11 — `concat` de múltiplos anos 50
Exercício 12 — Índice de tempo e fatiamento por período 50
Exercício 13 — `resample` mensal a partir de dados diários 50
Exercício 14 — Defasagem, diferença e variação percentual 50
Exercício 15 — Reconstruindo o IPCA acumulado em 12 meses 50

Exercício 16 — `std` do pandas vs. `numpy`, lado a lado 51
Exercício 17 — Matriz de correlação com `numeric_only` 51
Exercício 18 — Método encadeado (*method chaining*) 51
Exercício 19 — Exportando em dois formatos 51
Exercício 20 — `category` e uso de memória 51
Exercício 21 — Projeto integrador: painel BCB completo 51
Capítulo Final — Resumo 51
Capítulo 17 — Erros Comuns e Teste Rápido 52
17.1. Os erros que mais aparecem 52
17.2. Teste rápido: verdadeiro ou falso (com gabarito) 52
Próxima apostila 53

Como Usar Esta Apostila

Esta é a segunda apostila da série. Ela pressupõe a **Apostila 1** (listas, dicionários, funções, tratamento de erro) e ensina a trabalhar com **tabelas** — a estrutura em que quase todo dado econômico vive. Em cada capítulo, o conteúdo original ganha, ao final, um bloco de **aprofundamento** com exemplos *executados* (os resultados impressos vêm da execução do código) e, sempre que possível, **dados reais deste repositório**: a Selic meta diária (`selic_meta.json`), o IPCA acumulado em 12 meses (`ipca_12m.json`), a abertura do IPCA por grupos (`ipca_abertura_grupos.csv`) e a série da regra de Taylor (`taylor_rule_series.csv`). Rode os blocos na pasta do repositório para reproduzir.

Exemplo resolvido — modelo
Um exemplo executado e comentado.

Intuição
A ideia por trás da operação, sem jargão.

Aprofundando a teoria — modelo
O que acontece "por baixo": modelo mental preciso.

Atenção — erro comum
Um erro clássico (de iniciante e de veterano distraído).

No mercado brasileiro
Como o conceito aparece no trabalho de um economista.

Atenção — erro comum
Versão do pandas. Os resultados desta apostila foram gerados com o **pandas 2.2**. O pandas 3.0 mudou três comportamentos relevantes: (1) o **Copy-on-Write** passou a ser o padrão — a indexação encadeada deixa de funcionar e o `SettingWithCopyWarning` some (capítulo 5); (2) o tipo padrão de texto passou a ser um `str` dedicado em vez de `object`; (3) alguns apelidos de frequência (`"M"`, `"Q"`, `"A"`) foram substituídos por `"ME"`, `"QE"`, `"YE"`. Se seu resultado diferir em algum detalhe (tipo `str` no lugar de `object`, por exemplo), verifique com `pd.__version__`.

Capítulo 1 — O que é pandas e por que ele existe

1.1. Um pouco de história

pandas foi criado por Wes McKinney em 2008, enquanto ele trabalhava como analista quantitativo na AQR Capital Management, um fundo de investimentos. O problema que ele enfrentava era muito concreto: análise financeira e econômica exige manipular tabelas — cortar, filtrar, agrupar, juntar, alinhar séries temporais — e, na época, Python não tinha uma ferramenta boa para isso. Existia `numpy` para arrays numéricos, mas um array não tem noção de "coluna com nome" ou "linha identificada por uma data". McKinney construiu pandas exatamente para preencher essa lacuna, e o nome não é acidental: **pandas vem de "panel data"**, o termo econométrico para dados que combinam a dimensão temporal com a dimensão de corte transversal (ex: PIB de vários países ao longo de vários anos) — uma homenagem direta à origem do problema que a biblioteca resolve. A biblioteca foi liberada como código aberto em 2009 e hoje é, com folga, a ferramenta mais usada no mundo para análise de dados tabulares em Python — em bancos centrais, fundos de investimento, universidades e institutos de pesquisa.

1.2. pandas é construído sobre numpy — e isso importa

`numpy` é a biblioteca de computação numérica de Python: sua estrutura central é o `ndarray`, um array de tamanho fixo onde todos os elementos têm o **mesmo tipo** (todos inteiros, ou todos decimais, por exemplo) e ficam armazenados de forma contígua na memória. Essa uniformidade é o que permite ao numpy delegar operações inteiras a código compilado em C, evitando o laço `for` do Python (lento, como vimos na Apostila 1, Capítulo 10.4) elemento por elemento.

pandas herda diretamente essa arquitetura: cada coluna de um `DataFrame` é, por baixo dos panos, um array numpy (ou, em casos mais modernos, uma "extension array" — falaremos disso nos dtypes, Capítulo 3.3). Isso explica um comportamento central que confunde muitos iniciantes vindos do Excel: **operações em pandas são vetorizadas por padrão**. Somar duas colunas inteiras (`df["a"] + df["b"]`) não percorre linha por linha em Python — delega a soma inteira ao numpy, que a executa em C, elemento a elemento, de uma só vez. O ganho de velocidade para tabelas grandes (milhões de linhas, como o CADASTUR do BCB ou os microdados da PNAD) é de ordens de grandeza.

```
Excel/planilha            numpy                        pandas
──────────────────────────────────────────────────────────────────
Células soltas       →    array homogêneo        →     DataFrame
(qualquer tipo,           (1 tipo, contíguo,           (várias colunas numpy,
sem estrutura)             rápido em C)                 cada uma com Index e nome)
```

A relação não é apenas histórica — ela é prática todos os dias: sempre que você importa `import pandas as pd`, o numpy é importado junto (geralmente como `import numpy as np`), porque pandas depende dele internamente para quase toda operação numérica.

1.3. Os dois objetos centrais: Series e DataFrame

pandas oferece duas estruturas de dados principais:

- **`Series`** — um array unidimensional rotulado. Pense nele como **uma coluna de planilha com nome e índice**: cada valor tem, além da sua posição, um rótulo (o *Index*) que o identifica.
- **`DataFrame`** — uma estrutura bidimensional, com linhas e colunas rotuladas. Pense nele como **a planilha inteira**: uma coleção de `Series` que compartilham o mesmo Index (o mesmo "eixo de linhas").

```python
import pandas as pd

# Series: uma coluna
ipca = pd.Series([0.50, 0.70, 0.88, 0.67], name="ipca")
print(ipca)
```

Saída:

```
0    0.50
1    0.70
2    0.88
3    0.67
Name: ipca, dtype: float64
```

Repare na saída: à esquerda de cada valor aparece um número (`0`, `1`, `2`, `3`) — esse é o **Index**, e ele é criado automaticamente quando você não especifica um. `dtype: float64` informa o tipo de dado armazenado (voltaremos a isso no Capítulo 3.3).

Um `DataFrame`, por sua vez, é literalmente um dicionário de `Series` que compartilham o mesmo Index:

```python
dados = {
    "pais": ["Brasil", "Argentina", "Chile", "Colômbia"],
    "pib": [2.2, -1.8, 3.1, 2.8],
    "inflacao": [4.5, 98.0, 3.8, 7.2],
```

```
    "populacao": [214, 46, 19, 52],  # milhões
}

df = pd.DataFrame(dados)
print(df)
```

Saída:

```
        pais  pib  inflacao  populacao
0     Brasil  2.2       4.5        214
1  Argentina -1.8      98.0         46
2      Chile  3.1       3.8         19
3   Colômbia  2.8       7.2         52
```

Se você já entende dicionários e dicionários aninhados (Apostila 1, Capítulo 8), o `DataFrame` deve parecer familiar: `dados` era um dicionário `{coluna: lista_de_valores}`, e o `pd.DataFrame(...)` simplesmente organiza isso em uma tabela, adicionando um Index automático. Essa ponte conceitual é deliberada — pandas não inventa uma forma nova de pensar sobre dados, ele formaliza uma estrutura que você já usava informalmente com dicionários e listas.

1.4. O Index: o rótulo que a lista Python não tem

Esta é talvez a ideia mais estruturalmente importante de pandas, e a que menos aparece explicada nos tutoriais rápidos: **toda `Series` e todo `DataFrame` têm um Index**, e o Index não é "só uma coluna a mais" — é um objeto com papel funcional próprio.

O Index serve a três propósitos:

1. **Identificação de linha.** Em vez de acessar "a linha 5" (posição), você pode acessar "a linha do dia 2024-03-15" ou "a linha do Brasil" (rótulo). Isso é o que torna `df.loc["Brasil"]` possível.
2. **Alinhamento automático.** Quando você opera dois `DataFrames` ou `Series` juntos (soma, subtração), pandas primeiro **alinha pelos rótulos do Index**, não pela posição física. Isso é crucial e surpreende iniciantes: se `serie_a` tem índice `[2020, 2021, 2022]` e `serie_b` tem índice `[2021, 2022, 2023]`, `serie_a + serie_b` alinha por ano, e o resultado tem `NaN` (Capítulo 8) onde um dos dois não tinha dado — mesmo que as duas séries tenham o mesmo número de elementos.
3. **Performance de busca.** Um Index monotonicamente ordenado (por exemplo, datas em ordem crescente) permite que pandas localize um valor por busca binária ($O(\log n)$); um Index não ordenado usa uma tabela hash internamente, análoga à de um dicionário Python (Apostila 1, Capítulo 8.1) — em ambos os casos, muito mais rápido que percorrer a tabela inteira.

```
serie_a = pd.Series([2.9, 3.1, 2.2], index=[2022, 2023, 2024], name="pib_a")
serie_b = pd.Series([0.2, 2.6, 3.1], index=[2023, 2024, 2025], name="pib_b")

print(serie_a + serie_b)
```

Saída:

```
2022    NaN
2023    3.3
2024    5.3
2025    NaN
dtype: float64
```

Note que o resultado tem **quatro** anos (a união dos dois índices), não três — e os anos que só existiam em uma das séries viraram `NaN` (Not a Number, o marcador de dado ausente do pandas — Capítulo 8). Isso é o oposto do que aconteceria somando duas listas Python posicionalmente (`lista_a[i] + lista_b[i]`), que ignoraria

completamente os rótulos e simplesmente somaria por posição — produzindo um resultado silenciosamente errado se as séries não estivessem alinhadas na mesma ordem.

Por padrão, quando você não especifica um índice, pandas cria um `RangeIndex` — uma sequência `0, 1, 2, ...`, que é armazenada de forma extremamente eficiente em memória (não guarda cada número individualmente, só o início, fim e passo, como um `range()` do Python puro). Assim que você faz qualquer operação que reordene ou filtre as linhas, esse `RangeIndex` normalmente vira um `Index` genérico (guardando cada rótulo de fato) — o que explica por que, após um `sort_values()` ou um filtro, os números do índice "ficam fora de ordem" (ex: `2, 0, 3, 1`) até você chamar `reset_index()`.

1.5. Instalação e convenção de import

```
# pip install pandas numpy
import pandas as pd
import numpy as np
```

`pd` e `np` são apelidos (aliases) convencionais — não obrigatórios pela linguagem, mas seguidos universalmente pela comunidade (analogamente a `snake_case`, PEP 8, na Apostila 1). Usar esses apelidos não é apenas estética: qualquer economista que ler seu código, em qualquer lugar do mundo, reconhece `pd.DataFrame` e `np.array` instantaneamente.

1.6. Exemplo Resolvido — Alinhamento de Index em séries de fontes diferentes

Enunciado: você baixou o IPCA do BCB (código SGS 433) e, de outra fonte, uma projeção do Boletim Focus para os mesmos meses — mas a fonte do Focus só cobre até dois meses antes do fim da série do BCB (dado ainda não disponível). Ambas vêm como `Series` indexadas por mês (`"2024-01"`, `"2024-02"`, ...). Escreva o código que soma as duas séries e explique o que aparece nos meses finais, sem rodar no computador antes.

```
ipca_real = pd.Series(
    [0.42, 0.83, 0.56, 0.44, 1.20],
    index=["2024-01", "2024-02", "2024-03", "2024-04", "2024-05"],
    name="ipca_real",
)
foco_proj = pd.Series(
    [0.40, 0.75, 0.60],
    index=["2024-01", "2024-02", "2024-03"],
    name="foco_proj",
)
```

Solução comentada:

```
soma = ipca_real + foco_proj
print(soma)
```

Saída:

```
2024-01    0.82
2024-02    1.58
2024-03    1.16
2024-04     NaN
2024-05     NaN
dtype: float64
```

O Index de `soma` é a **união** dos dois índices (`2024-01` a `2024-05`), porque pandas alinha por rótulo antes de somar (Capítulo 1.4), não por posição. Nos meses `2024-04` e `2024-05`, `foco_proj` simplesmente não tem uma linha correspondente — e, em vez de um erro, pandas produz `NaN`: "não sei somar um número com a ausência de um número". Isso é uma diferença de comportamento importante frente a uma lista Python comum, onde `lista_a[i] + lista_b[i]` sequer rodaria se as listas tivessem tamanhos diferentes (`IndexError`), e rodaria

silenciosamente errado se tivessem o mesmo tamanho mas ordens diferentes. O alinhamento por Index é uma rede de segurança — mas também significa que você deve **sempre verificar se apareceram `NaN` inesperados** depois de somar duas séries de fontes diferentes, tratando-os com as ferramentas do Capítulo 8.

> **Aprofundando a teoria — por que pandas é rápido: arrays tipados, vetorização e broadcasting**
> Uma lista Python guarda *referências* a objetos espalhados na memória; um array NumPy guarda os valores **contíguos** e de **um único tipo** (`dtype`). Operar sobre o array inteiro de uma só vez (**vetorização**) delega o laço a código compilado, centenas de vezes mais rápido que o `for` do Python. O **broadcasting** estende operações entre arrays de formatos diferentes (por exemplo, subtrair a média de cada coluna). O argumento `axis` indica *em que direção* operar: `axis=0` percorre as *linhas* (resulta uma linha por coluna); `axis=1`, as *colunas*. Todo `Series` é um array com rótulos; todo `DataFrame`, um conjunto de colunas-array de mesmo comprimento.

```
precos = np.array([100.0, 104.0, 109.2, 113.6])
deflator = np.array([1.00, 1.02, 1.05, 1.08])
print(f"ndarray de 1 milhão de inteiros: {np.arange(1_000_000).nbytes:,} bytes (dtype
{np.arange(3).dtype})")
print("Preços reais, vetorizado (sem laço):", np.round(precos / deflator, 2))

m = np.array([[1, 2, 3], [4, 5, 6]])
print("soma axis=0 (uma por coluna):", m.sum(axis=0), "| axis=1 (uma por linha):",
m.sum(axis=1))
print("broadcasting (matriz - média da coluna):")
print(m - m.mean(axis=0))
```

```
ndarray de 1 milhão de inteiros: 8,000,000 bytes (dtype int64)
Preços reais, vetorizado (sem laço): [100.   101.96 104.   105.19]
soma axis=0 (uma por coluna): [5 7 9] | axis=1 (uma por linha): [ 6 15]
broadcasting (matriz - média da coluna):
[[-1.5 -1.5 -1.5]
 [ 1.5  1.5  1.5]]
```

> **Atenção — erro comum**
> **Operações entre tipos diferentes alteram o `dtype`.** Uma coluna de inteiros com um único `NaN` vira `float64`, porque `NaN` é um float; somar `True`/`False` conta verdadeiros. Quando o resultado de uma conta parecer estranho, a primeira checagem é `.dtype`.

Capítulo 2 — Criando DataFrames e lendo dados reais

2.1. A partir de estruturas Python

```
import pandas as pd

# A partir de um dicionário de listas (uma chave = uma coluna)
dados = {
    "pais": ["Brasil", "Argentina", "Chile", "Colômbia"],
    "pib": [2.2, -1.8, 3.1, 2.8],
}
df = pd.DataFrame(dados)

# A partir de uma lista de dicionários (um dicionário = uma linha)
registros = [
    {"pais": "Brasil", "pib": 2.2},
    {"pais": "Argentina", "pib": -1.8},
```

```
]
df2 = pd.DataFrame(registros)

# A partir de listas de listas, com nomes de coluna explícitos
linhas = [["Brasil", 2.2], ["Argentina", -1.8]]
df3 = pd.DataFrame(linhas, columns=["pais", "pib"])
```

A escolha entre "dicionário de listas" (orientado a coluna) e "lista de dicionários" (orientado a linha) geralmente é ditada pelo formato em que o dado chega até você: uma API que devolve um registro por vez (uma linha, com vários campos) naturalmente produz uma lista de dicionários; um arquivo já tabular naturalmente produz um dicionário de colunas.

2.2. `read_csv` em detalhe — a função mais usada de pandas

```
df = pd.read_csv("dados.csv")
```

Na prática profissional, quase nunca é tão simples assim — arquivos CSV reais, principalmente os exportados de sistemas brasileiros (Excel em português, sistemas legados de órgãos públicos), têm particularidades que `read_csv` precisa ser instruído a tratar:

Parâmetro	Para que serve	Exemplo típico no Brasil
`sep`	Caractere separador de colunas	`sep=";"` — Excel BR usa `;`, não `,`, porque `,` já é o separador decimal
`decimal`	Caractere separador decimal	`decimal=","` — "3,14" em vez de "3.14"
`encoding`	Codificação de caracteres	`encoding="latin1"` ou `"cp1252"` — arquivos exportados de sistemas antigos do governo raramente estão em UTF-8
`thousands`	Separador de milhar	`thousands="."` — "1.234.567"
`parse_dates`	Colunas a converter para data já na leitura	`parse_dates=["data"]`
`dayfirst`	Interpreta datas ambíguas como dia/mês/ano	`dayfirst=True` — "01/02/2024" = 1º de fevereiro, não 2 de janeiro
`na_values`	Strings adicionais a tratar como nulo	`na_values=["-", "..", "ND", "N/D"]`
`skiprows`	Pula linhas do topo (cabeçalhos extras, notas)	`skiprows=3`
`usecols`	Lê só um subconjunto de colunas	`usecols=["data", "valor"]`
`dtype`	Força o tipo de uma coluna na leitura	`dtype={"codigo_ibge": str}` — evita que um código vire número e perca zeros à esquerda

```
df = pd.read_csv(
    "ipca_ibge.csv",
    sep=";",
    decimal=",",
    encoding="latin1",
    parse_dates=["data"],
    dayfirst=True,
    na_values=["-", "..", "ND"],
)
```

Armadilha comum: um código de município do IBGE como `"03550308"` (São Paulo) tem um zero à esquerda que carrega informação. Se `read_csv` não for instruído com `dtype={"codigo_ibge": str}`, pandas infere automaticamente que a coluna é numérica e **descarta o zero à esquerda silenciosamente**, virando `3550308`

— um erro sutil que só aparece quando você tenta cruzar (merge) essa coluna com outra fonte que preservou o zero.

2.3. `read_excel`

```
df = pd.read_excel("dados.xlsx", sheet_name="Planilha1")

# Múltiplas abas de uma vez (retorna um dicionário {nome_aba: DataFrame})
todas_abas = pd.read_excel("dados.xlsx", sheet_name=None)

# Pular linhas de cabeçalho/nota (comum em planilhas do BCB/IBGE, que têm
# título e fonte antes da tabela de verdade)
df = pd.read_excel("serie_bcb.xlsx", skiprows=4, skipfooter=2)
```

`read_excel` exige o pacote `openpyxl` instalado (para `.xlsx`) por baixo dos panos — pandas delega a leitura do formato binário/XML da planilha a essas bibliotecas especializadas, e só organiza o resultado em um `DataFrame`.

2.4. Fontes de dados econômicos brasileiros

Um economista raramente digita dados manualmente — ele os baixa de uma API ou repositório público. As três fontes mais usadas no Brasil, e como elas aparecem em Python:

Fonte	O que oferece	Biblioteca Python	Exemplo
BCB SGS (Sistema Gerenciador de Séries Temporais)	+25.000 séries temporais: IPCA, SELIC, câmbio, crédito, etc.	`python-bcb` (`from bcb import sgs`)	`sgs.get({"IPCA": 433}, start="2010-01-01")`
Ipeadata	Séries do IBGE, BCB e do próprio IPEA compiladas em um só lugar, incluindo contas nacionais	`ipeadatapy`	`ipeadatapy.timeseries("PRECOS12_IPCA12")`
IBGE SIDRA	Censo, PNAD Contínua, Contas Nacionais Trimestrais, microdados agregados	`sidrapy`	`sidrapy.get_table(table_code="1737", ...)`

Cada uma dessas bibliotecas devolve, ao final, um `pandas.DataFrame` (ou `Series`) — é por isso que dominar pandas é o pré-requisito comum a qualquer uma dessas fontes: a biblioteca de acesso muda, a forma de manipular o resultado, não.

```
from bcb import sgs

# Código 433 = IPCA (variação mensal, %); código 11 = SELIC (taxa diária, % a.a.)
ipca = sgs.get({"IPCA": 433}, start="2020-01-01")
selic = sgs.get({"SELIC": 11}, start="2020-01-01")

print(ipca.head())
print(type(ipca))   # <class 'pandas.core.frame.DataFrame'>
```

2.5. Outras origens

```
# Área de transferência — copie uma tabela do Excel e rode:
df = pd.read_clipboard()

# Dicionário Python já em memória
df = pd.DataFrame.from_dict({"a": [1, 2], "b": [3, 4]})

# Lista de tuplas/registros com nomes de campo
df = pd.DataFrame.from_records([(1, "Brasil"), (2, "Chile")], columns=["id", "pais"])
```

```python
# JSON (comum em respostas de API)
df = pd.read_json("dados.json")
```

2.6. Exemplo Resolvido — Importando um CSV "sujo" do jeito certo

Enunciado: você recebeu o arquivo `pib_estados.csv`, exportado de um sistema legado, com o seguinte conteúdo (mostrado aqui como texto para fins didáticos):

```
Fonte: Sistema Legado Estadual - Uso Interno
Gerado em: 15/03/2024
uf;pib_r_milhoes;populacao
SP;2.450.123,50;46.000.000
RJ;890.456,20;17.500.000
MG;620.789,10;21.400.000
```

Escreva a chamada de `pd.read_csv` que lê esse arquivo corretamente, resultando em colunas numéricas de verdade (não texto).

Solução comentada:

```python
df = pd.read_csv(
    "pib_estados.csv",
    sep=";",           # separador é ponto e vírgula, não vírgula
    decimal=",",       # a vírgula em "2.450.123,50" é o separador decimal
    thousands=".",     # o ponto em "2.450.123" é separador de milhar
    skiprows=2,        # pula as duas linhas de "Fonte:" e "Gerado em:"
    encoding="latin1", # sistemas legados raramente exportam em UTF-8
)

print(df)
print(df.dtypes)
```

Saída esperada:

```
   uf  pib_r_milhoes  populacao
0  SP     2450123.50   46000000
1  RJ      890456.20   17500000
2  MG      620789.10   21400000

uf                object
pib_r_milhoes    float64
populacao          int64
dtype: object
```

O raciocínio segue a mesma lógica de "limpar antes de converter" vista na Apostila 1 (Capítulo 4.6, conversão de `"R$ 1.234,56"`): primeiro identificamos o que **não é** um número válido em formato americano (separador de milhar `.` e decimal `,`, invertidos frente ao padrão que `float()`/pandas esperam), e instruímos `read_csv` a desfazer isso na leitura, em vez de importar tudo como texto e corrigir depois — mais rápido e menos propenso a erro. Repare também que `skiprows=2` descarta as duas primeiras linhas de metadado, um padrão extremamente comum em planilhas de órgãos públicos brasileiros, que costumam colocar título e fonte acima da tabela real.

Exemplo resolvido — laboratório de `read_csv`: um arquivo "à brasileira" lido do jeito certo

Arquivos brasileiros costumam usar `;` como separador, vírgula decimal, ponto de milhar, datas `dd/mm/aaaa` e marcadores de ausente como `N/D`. Cada convenção tem um parâmetro:

Parâmetro	Para quê
`sep=";"`	Separador de colunas

Parâmetro	Para quê
`decimal=","` e `thousands="."`	Números no padrão brasileiro
`parse_dates=[...]` e `dayfirst=True`	Converter colunas em datas, com dia antes do mês
`na_values=["N/D", "-"]`	Textos que significam "ausente"
`usecols=[...]`, `nrows=N`	Ler só algumas colunas ou linhas (arquivos grandes)
`dtype={"col": "string"}`	Forçar o tipo de uma coluna
`encoding="latin-1"`	Arquivos antigos em codificação não UTF-8
`skiprows=N`, `header=...`	Pular linhas de título e escolher o cabeçalho

```
from io import StringIO
csv_br = """Data;Estado;PIB (R$ mi);Obs
31/03/2025;SP;"2.345,6";ok
31/03/2025;RJ;"1.020,5";
30/06/2025;SP;"2.401,0";revisado
30/06/2025;RJ;N/D;ok"""
ingenuo = pd.read_csv(StringIO(csv_br))
correto = pd.read_csv(StringIO(csv_br), sep=";", decimal=",", thousands=".",
parse_dates=["Data"],
                      dayfirst=True, na_values=["N/D"], dtype={"Estado": "string"})
print("Leitura ingênua -> colunas:", list(ingenuo.columns))
print(correto)
print(correto.dtypes)
```

```
Leitura ingênua -> colunas: ['Data;Estado;PIB (R$ mi);Obs']
        Data Estado  PIB (R$ mi)       Obs
0 2025-03-31     SP       2345.6        ok
1 2025-03-31     RJ       1020.5       NaN
2 2025-06-30     SP       2401.0  revisado
3 2025-06-30     RJ          NaN        ok
Data           datetime64[ns]
Estado         string[python]
PIB (R$ mi)           float64
Obs                    object
dtype: object
```

Exemplo resolvido — lendo um JSON de API (a Selic meta diária do Banco Central)
Séries do BCB chegam como lista de registros com a data (`"dd/mm/aaaa"`) e o valor como *texto*. O padrão é: ler, converter a data (indicando o formato), converter o valor para número e usar a data como **índice**:

```
selic_d = pd.read_json("selic_meta.json", dtype=False)
selic_d["data"] = pd.to_datetime(selic_d["data"], format="%d/%m/%Y")
selic_d["valor"] = pd.to_numeric(selic_d["valor"])
selic_d = selic_d.set_index("data")["valor"].rename("selic_meta")
print(f"{len(selic_d)} dias | de {selic_d.index.min():%d/%m/%Y} a {selic_d.index.max():%d/
%m/%Y}")
print(selic_d.tail(3))

ipca12 = pd.read_json("ipca_12m.json", dtype=False)
ipca12["data"] = pd.to_datetime(ipca12["data"], format="%d/%m/%Y")
```

```
ipca12 = ipca12.set_index("data")["valor"].astype(float).rename("ipca_12m")
print(f"IPCA 12m: {len(ipca12)} meses | último: {ipca12.iloc[-1]:.2f}% em
{ipca12.index[-1]:%m/%Y}")
```

```
1355 dias | de 01/01/2023 a 16/09/2026
data
2026-09-14    14.0
2026-09-15    14.0
2026-09-16    14.0
Name: selic_meta, dtype: float64
IPCA 12m: 43 meses | último: 4.44% em 07/2026
```

Capítulo 3 — Explorando e entendendo a estrutura dos dados

3.1. Primeiro contato com uma tabela nova

Antes de qualquer análise, o primeiro hábito profissional é **nunca confiar cegamente** em um DataFrame recém-importado — sempre inspecioná-lo:

```
df.head()        # primeiras 5 linhas
df.head(10)      # primeiras 10
df.tail(3)       # últimas 3 (útil para ver se a série termina onde deveria)
df.sample(5)     # 5 linhas aleatórias (útil para pegar problemas "no meio" da tabela)

df.shape         # (número de linhas, número de colunas) — uma tupla
df.info()        # tipos de dados + contagem de não-nulos + uso de memória
df.describe()    # estatísticas descritivas das colunas numéricas
df.dtypes        # tipo de cada coluna
df.columns       # nomes das colunas (um Index de strings)
df.index         # o Index das linhas
```

`df.info()` merece destaque: em uma única chamada, ele revela três coisas que costumam expor problemas de importação — (1) se o número de linhas "não-nulas" bate com o total esperado (revelando dados faltantes), (2) se o `dtype` de cada coluna é o esperado (uma coluna de datas lida como `object`, por exemplo, é sinal de que `parse_dates` deveria ter sido usado), e (3) o uso de memória do DataFrame, relevante para bases muito grandes.

```
print(df.info())
```

Saída típica:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   pais       4 non-null      object
 1   pib        4 non-null      float64
 2   inflacao   4 non-null      float64
 3   populacao  4 non-null      int64
dtypes: float64(2), int64(1), object(1)
memory usage: 260.0+ bytes
```

3.2. `describe()`, `unique()`, `value_counts()`

```
df.describe()               # count, mean, std, min, 25%, 50%, 75%, max — só colunas
numéricas
df.describe(include="all")  # inclui colunas de texto (mostra unique, top, freq)

df["pais"].unique()          # array com os valores distintos, na ordem em que aparecem
df["pais"].nunique()         # quantos valores distintos existem
df["pais"].value_counts()    # contagem de cada valor, ordenado do mais frequente ao menos
df["pais"].value_counts(normalize=True)  # o mesmo, mas em proporção (soma 1.0)
```

`value_counts()` é, na prática, uma das ferramentas de diagnóstico mais usadas no dia a dia: rodá-la sobre uma coluna categórica logo após importar um arquivo revela imediatamente problemas de digitação (`"Brasil"` vs `"brasil"` vs `"BRASIL"` contados como três categorias diferentes) — um sintoma comum de dados digitados manualmente.

3.3. Tipos de dados (dtypes) do pandas — mais do que parece

Um `dtype` (data type) descreve como os bytes de uma coluna são interpretados na memória. Entender os principais evita bugs sutis (o próximo capítulo, sobre `NaN`, depende diretamente disso):

dtype	Representa	Exemplo	Observação
`int64`	Inteiro de 64 bits	`populacao`	Não aceita `NaN` — se um `NaN` entrar, a coluna inteira é promovida a `float64`
`float64`	Decimal de ponto flutuante de 64 bits	`pib`, `inflacao`	Aceita `NaN` nativamente (é o tipo padrão para colunas numéricas com dados faltantes)
`bool`	Verdadeiro/Falso	`em_recessao`	Também não aceita `NaN` nativamente (mesma lógica do `int64`)
`object`	Texto (`str`) ou tipos mistos	`pais`, `sigla`	O "tipo genérico" — qualquer coluna que não seja puramente numérica cai aqui, mesmo que só contenha texto
`category`	Texto com um conjunto **fixo e pequeno** de valores possíveis	`regiao` (Norte, Nordeste, ...)	Muito mais eficiente em memória que `object` quando há poucos valores únicos repetidos muitas vezes
`datetime64[ns]`	Data/hora	`data`	Permite aritmética de datas e o acessor `.dt` (Capítulo 11)
`timedelta64[ns]`	Duração (diferença entre datas)	`data_fim - data_inicio`	Resultado de subtrair duas colunas `datetime64`
`Int64` (nullable, "I" maiúsculo)	Inteiro que **aceita** `NaN`/`pd.NA`	Introduzido no pandas 1.0	Diferente de `int64` (minúsculo) — resolve o problema de upcast a `float`

```
df.dtypes
# pais          object
# pib          float64
# inflacao     float64
# populacao      int64
# dtype: object

# Convertendo o tipo de uma coluna
df["populacao"] = df["populacao"].astype("int64")
```

```
df["regiao"] = df["regiao"].astype("category")   # economiza memória se houver poucas
regiões repetidas
```

Por que `object` é o dtype "genérico" e por que isso é lento. Quando uma coluna é `object`, cada célula é, na prática, um ponteiro Python para um objeto `str` (ou qualquer outro tipo) separado na memória — exatamente como uma lista Python comum, e não como um array numpy contíguo de números. Isso significa que operações em colunas `object` **não se beneficiam da vetorização em C** da mesma forma que colunas numéricas — são mais lentas e usam mais memória. Converter uma coluna de texto repetitivo (como uma sigla de país ou região) para `category` recupera parte dessa eficiência, porque `category` armazena internamente só os valores únicos (o "dicionário" de categorias) e um array de códigos inteiros apontando para eles — a mesma ideia de um dicionário Python (Apostila 1, Capítulo 8.1) aplicada a uma coluna inteira.

3.4. Exemplo Resolvido — Diagnosticando um DataFrame recém-importado

Enunciado: você rodou `pd.read_csv("serie_bcb.csv")` sem nenhum parâmetro extra e obteve o `df.info()` abaixo. Aponte, usando só essa saída, dois prováveis problemas de importação e o parâmetro de `read_csv` que resolveria cada um.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 300 entries, 0 to 299
Data columns (total 3 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   data     300 non-null    object
 1   ipca     287 non-null    object
 2   selic    300 non-null    float64
dtypes: float64(1), object(2)
memory usage: 7.2+ KB
```

Solução comentada:

1. **A coluna `data` está como `object`, não `datetime64[ns]`.** Isso indica que `read_csv` não recebeu `parse_dates=["data"]`. Sem essa conversão, você não consegue usar o acessor `.dt` (Capítulo 11) nem fatiar por período com `.loc["2020":"2021"]`.
2. **A coluna `ipca` está como `object` (texto), mas deveria ser numérica** — e ainda por cima tem só 287 valores não-nulos em 300 linhas, contra 300/300 de `selic`. A causa mais provável: a coluna `ipca` contém, em algumas linhas, um marcador de dado ausente que `read_csv` não reconheceu como nulo (por exemplo, `"-"` ou `"ND"`), ou usa vírgula decimal sem `decimal=","` especificado — em ambos os casos, pandas não consegue interpretar a coluna inteira como número e a mantém como texto (`object`). A correção:

```
df = pd.read_csv(
    "serie_bcb.csv",
    parse_dates=["data"],
    decimal=",",
    na_values=["-", "ND", ".."],
)
print(df.dtypes)
# data     datetime64[ns]
# ipca            float64
# selic           float64
```

O ponto central deste exemplo: **`df.info()` é a primeira coisa a rodar depois de qualquer importação**, porque um `dtype` errado raramente gera um erro imediato — ele se manifesta silenciosamente, mais tarde, como um `TypeError` ao tentar somar a coluna, ou como um resultado numericamente sem sentido (comparar strings como se fossem números compara por ordem alfabética, não por valor).

Exemplo resolvido — receita de diagnóstico de uma base nova (abertura do IPCA por grupos)
Antes de qualquer análise, a rotina de cinco comandos: *formato, tipos, memória, resumo estatístico e nulos.* Aplicada ao arquivo `ipca_abertura_grupos.csv` (acumulado em 12 meses por grupo de preços):

```
grupos = pd.read_csv("ipca_abertura_grupos.csv", parse_dates=["data"], index_col="data")
print("Formato:", grupos.shape, "| período:", grupos.index.min().strftime("%m/%Y"), "a",
grupos.index.max().strftime("%m/%Y"))
grupos.info()
print("\nResumo estatístico:")
print(grupos.describe().round(2).T[["mean", "std", "min", "max"]].to_string())
print(f"\nNulos por coluna: {grupos.isna().sum().to_dict()}")
print(f"Memória: {grupos.memory_usage(deep=True).sum():,} bytes")
```

```
Formato: (43, 6) | período: 01/2023 a 07/2026
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 43 entries, 2023-01-01 to 2026-07-01
Data columns (total 6 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Servicos              43 non-null     float64
 1   Administrados         43 non-null     float64
 2   Livres                43 non-null     float64
 3   Comercializaveis      43 non-null     float64
 4   Nao-comercializaveis  43 non-null     float64
 5   IPCA cheio            43 non-null     float64
dtypes: float64(6)
memory usage: 2.4 KB

Resumo estatístico:
                      mean   std   min    max
Servicos              5.78  0.81  4.47   7.85
Administrados         4.91  3.24 -2.79  10.23
Livres                4.54  1.38  2.84   8.93
Comercializaveis      3.80  2.33  0.53   9.06
Nao-comercializaveis  5.11  1.06  4.03   8.86
IPCA cheio            4.61  0.56  3.16   5.77

Nulos por coluna: {'Servicos': 0, 'Administrados': 0, 'Livres': 0, 'Comercializaveis': 0,
'Nao-comercializaveis': 0, 'IPCA cheio': 0}
Memória: 2,408 bytes
```

Aprofundando a teoria — tipos do pandas: `object`, `category`, tipos anuláveis e economia de memória
Colunas de texto vêm como `object` (referências a strings Python), que ocupam muita memória e são lentas. Se a coluna tem **poucos valores distintos repetidos** (região, setor, rating), o tipo `category` guarda cada valor distinto *uma vez* e um código inteiro por linha — grande economia e ordenação lógica possível. Os tipos **anuláveis** (`Int64`, `boolean`, `string`) aceitam ausentes sem converter para `float` (inteiros com `NaN` viram `float64` nos tipos clássicos). `convert_dtypes()` escolhe automaticamente os tipos mais adequados.

```
rating = pd.Series(["AAA", "AA", "A", "BBB"] * 25_000, name="rating")
rating_cat = rating.astype("category")
print(f"object   : {rating.memory_usage(deep=True):>10,} bytes")
print(f"category : {rating_cat.memory_usage(deep=True):>10,} bytes ({1 -
rating_cat.memory_usage(deep=True) / rating.memory_usage(deep=True):.0%} menos)")

com_nulo = pd.Series([1, 2, None, 4])
print("Série com ausente:", com_nulo.dtype, "| com tipo anulável:",
com_nulo.astype("Int64").dtype, "->", com_nulo.astype("Int64").tolist())
```

```
object   :  5,125,132 bytes
category :    100,509 bytes (98% menos)
Série com ausente: float64 | com tipo anulável: Int64 -> [1, 2, <NA>, 4]
```

Capítulo 4 — Seleção de dados: `[]`, `.loc`, `.iloc`, `.at`, `.iat`

4.1. Selecionando colunas

```
df["pais"]              # uma coluna → retorna Series
df.pais                  # atalho de atributo — funciona só se o nome não tiver espaço,
                         # não colidir com um método existente (ex: df.count) e for um
                         # identificador Python válido; em código de produção, prefira
df["pais"]
df[["pais", "pib"]]      # várias colunas, passadas como LISTA → retorna DataFrame (repare
nos dois colchetes)
```

O atalho `df.pais` é conveniente no terminal interativo, mas tem armadilhas reais: se uma coluna se chamar `"count"` ou `"mean"`, `df.count` acessa o **método** `count()` do DataFrame, não a coluna — um bug silencioso, já que não gera erro, apenas devolve o objeto errado. Por essa razão, código destinado a ser reaproveitado ou revisado por terceiros deve usar sempre `df["coluna"]`.

4.2. `.iloc` — seleção por posição (i de "integer location")

```
df.iloc[0]         # primeira linha (posição 0), como Series
df.iloc[-1]        # última linha
df.iloc[0:3]       # linhas nas posições 0, 1, 2 (o fim, 3, NÃO é incluído)
df.iloc[[0, 2]]    # linhas nas posições 0 e 2 (lista explícita de posições)

df.iloc[0, 1]      # escalar: linha na posição 0, coluna na posição 1
df.iloc[0:3, 0:2]  # sublinha/subcoluna: linhas 0-2, colunas 0-1
df.iloc[:, 0]      # todas as linhas, primeira coluna
```

`.iloc` segue exatamente a mesma convenção de fatiamento (slicing) de listas Python vista na Apostila 1 (Capítulo 7.2): o índice final é **exclusivo**. `df.iloc[0:3]` traz 3 linhas (posições 0, 1, 2), não 4.

4.3. `.loc` — seleção por rótulo ("location" pelo Index)

```
df.index = ["a", "b", "c", "d"]

df.loc["a"]          # a linha cujo ÍNDICE (rótulo) é "a" — não a primeira posição
df.loc["a":"c"]      # linhas de "a" até "c" — repare: "c" está INCLUÍDO!
df.loc[:, "pib"]     # todas as linhas, coluna "pib"
df.loc["a", "pib"]   # escalar: linha "a", coluna "pib"
df.loc[["a", "c"]]   # linhas "a" e "c" (lista explícita de rótulos)

# .loc também aceita uma máscara booleana — é a base do filtro do Capítulo 6
df.loc[df["pib"] > 2]
```

4.4. A diferença crucial entre `.loc` e `.iloc`: inclusivo vs. exclusivo

Esta é a armadilha mais comum de quem começa com pandas, e vale destacá-la isoladamente:

	`.iloc` (posição)	`.loc` (rótulo)
Índice usado	Posição inteira (0, 1, 2, ...), sempre, independente do Index real	O rótulo do Index, seja ele número, string ou data

	`.iloc` (posição)	`.loc` (rótulo)
Fim do fatiamento (`a:b`)	**Exclusivo** — `b` não entra, igual a uma lista Python	**Inclusivo** — `b` entra!
`df.iloc[0:3]`	3 linhas (posições 0, 1, 2)	—
`df.loc["a":"c"]` (índice = a,b,c,d)	—	3 linhas ("a", "b", "c" — "c" está incluído)
Funciona se o Index não for numérico?	Sim (posição sempre existe)	Sim, é o caso de uso natural
Funciona se o Index for numérico mas fora de ordem (ex: `[3, 1, 2]`)?	Sim, ignora o Index, usa só a posição física	Sim, mas `.loc[1:2]` busca os *rótulos* 1 e 2, não as posições 1 e 2!

A razão para `.loc` incluir o rótulo final (diferente de toda convenção de fatiamento do restante do Python) é justamente por ser baseado em **rótulo**, não posição: não existe uma noção geral de "o próximo rótulo depois de 'c'" para todo tipo de Index (ele poderia ser uma data, uma string, um número não sequencial) — então incluir o limite final é a única forma consistente de garantir que `df.loc["2020-01":"2020-06"]` traga o mês de junho inteiro, e não pare "um pouco antes" dele.

```
serie = pd.Series([10, 20, 30, 40], index=[3, 1, 2, 0])
print(serie.iloc[0:2])   # posições 0 e 1 → valores 10 e 20 (rótulos 3 e 1)
print(serie.loc[0:2])    # ATENÇÃO: com índice não ordenado, .loc[0:2] pode
                         # até lançar erro (KeyError) ou se comportar de forma
                         # não intuitiva — .loc espera, idealmente, um índice ordenado
                         # para fatiamento por intervalo
```

4.5. `.at` e `.iat` — acesso a um único valor, mais rápido

```
df.at["a", "pib"]     # equivalente a df.loc["a", "pib"], mas otimizado para UM valor
escalar
df.iat[0, 1]          # equivalente a df.iloc[0, 1], mesma otimização
```

Para acessar um valor único, `.at`/`.iat` evitam parte da sobrecarga interna de `.loc`/`.iloc` (que são generalizados para lidar com fatias, listas e máscaras). Em um laço que acessa milhares de células individualmente (embora, como veremos no Capítulo 7, um laço explícito raramente é a melhor abordagem em pandas), `.at`/`.iat` são mensuravelmente mais rápidos.

4.6. Exemplo Resolvido — Escolhendo a ferramenta certa de seleção

Enunciado: dado o DataFrame abaixo, indexado por sigla de país, escreva a expressão de seleção mais apropriada (usando `.loc`, `.iloc`, `.at` ou colchetes simples) para cada uma das quatro tarefas.

```
df = pd.DataFrame(
    {"pib": [2.2, -1.8, 3.1], "inflacao": [4.5, 98.0, 3.8]},
    index=["BRA", "ARG", "CHL"],
)
```

a) Obter só a coluna `inflacao` inteira.
b) Obter a linha do Chile.
c) Obter apenas o valor do PIB da Argentina (um único número).
d) Obter a segunda linha da tabela, seja qual for seu rótulo.

Solução comentada:

```
# a) coluna inteira -> colchetes simples, sem .loc necessário
print(df["inflacao"])

# b) linha por RÓTULO (sigla do país) -> .loc
print(df.loc["CHL"])
```

```
# c) um único valor, por rótulo -> .at é o mais direto e rápido
print(df.at["ARG", "pib"])   # -1.8

# d) linha por POSIÇÃO, independente do rótulo -> .iloc
print(df.iloc[1])            # segunda linha = ARG, aqui coincide com (c), mas por outro
caminho
```

O critério de decisão é sempre o mesmo: **pergunte-se se você está pensando em "onde fisicamente" (posição) ou "o quê" (identificador/rótulo)**. Tarefa (d) pede explicitamente "a segunda linha, seja qual for seu rótulo" — uma noção de posição, portanto `.iloc`. Tarefas (b) e (c) pedem por identidade (o país "Chile", a "Argentina") — noção de rótulo, portanto `.loc`/`.at`. Usar `.iloc[1]` para a tarefa (b) funcionaria *neste exemplo específico*, porque o Chile por acaso está na posição 2 — mas seria um código frágil, que quebraria silenciosamente se a ordem das linhas mudasse (por exemplo, após um `sort_values`).

Exemplo resolvido — seleção por rótulo e por posição em uma série temporal real
Com a Selic meta diária indexada por data, a diferença entre `.loc` (rótulos, extremo *inclusivo*) e `.iloc` (posições, extremo *exclusivo*) fica concreta — e o fatiamento por data aceita strings parciais:

```
print("loc  ('2025-12-30':'2026-01-02'), os dois extremos incluídos:")
print(selic_d.loc["2025-12-30":"2026-01-02"].to_string())
print("\niloc [0:3], três primeiras posições (fim exclusivo):")
print(selic_d.iloc[0:3].to_string())
print("\nFatiamento por ano inteiro e por mês:", len(selic_d.loc["2025"]), "dias em 2025
|", len(selic_d.loc["2026-03"]), "dias em mar/2026")
print("Valor num único dia (.at):", selic_d.at[pd.Timestamp("2026-09-16")])

mudancas = selic_d[selic_d.diff().fillna(0) != 0]
print(f"\nA meta mudou {len(mudancas)} vezes no período; as últimas:")
print(mudancas.tail(4).to_string())
print(f"Máxima: {selic_d.max()}% (primeira vez em {selic_d.idxmax():%d/%m/%Y}) | mínima:
{selic_d.min()}%")
```

```
loc  ('2025-12-30':'2026-01-02'), os dois extremos incluídos:
data
2025-12-30    15.0
2025-12-31    15.0
2026-01-01    15.0
2026-01-02    15.0

iloc [0:3], três primeiras posições (fim exclusivo):
data
2023-01-01    13.75
2023-01-02    13.75
2023-01-03    13.75

Fatiamento por ano inteiro e por mês: 365 dias em 2025 | 31 dias em mar/2026
Valor num único dia (.at): 14.0

A meta mudou 18 vezes no período; as últimas:
data
2026-03-19    14.75
2026-04-30    14.50
2026-06-18    14.25
2026-08-06    14.00
Máxima: 15.0% (primeira vez em 19/06/2025) | mínima: 10.5%
```

Capítulo 5 — Cópia vs. view: o `SettingWithCopyWarning`

5.1. Por que este capítulo existe

Na Apostila 1 (Capítulo 7.3), vimos que listas Python são mutáveis e que atribuir uma lista a outra variável não cria uma cópia — cria um segundo rótulo apontando para o mesmo objeto (*aliasing*). pandas tem um problema **irmão** desse, só que mais traiçoeiro, porque ele não é binário (cópia ou não-cópia) — depende de detalhes internos de implementação que nem sempre são visíveis a partir do código-fonte. O sintoma é um aviso que todo economista que usa pandas encontra eventualmente:

```
SettingWithCopyWarning: A value is trying to be set on a copy of a slice from a DataFrame.
```

5.2. Reproduzindo o problema

```
df = pd.DataFrame({"pais": ["Brasil", "Argentina", "Chile"], "pib": [2.2, -1.8, 3.1]})

# "Indexação encadeada" (chained indexing) — dois passos de seleção em sequência
positivos = df[df["pib"] > 0]     # passo 1: filtra (pode devolver view OU cópia — pandas
não garante)
positivos["situacao"] = "cresceu" # passo 2: tenta atribuir na "fatia" resultante
# SettingWithCopyWarning!
```

O problema: `df[df["pib"] > 0]` **pode** devolver uma nova cópia independente de `df`, ou **pode** devolver uma *view* (uma "janela" que aponta para os mesmos dados de `df`) — e isso depende de detalhes internos (o tipo de operação, o dtype das colunas envolvidas) que o próprio pandas não expõe como garantia estável entre versões. Quando você faz `positivos["situacao"] = ...` sobre o resultado, pandas não consegue garantir se essa atribuição:

- alterou de fato `df` original (porque `positivos` era uma *view*), ou
- criou a coluna só em `positivos`, e `df` original permaneceu intocado (porque `positivos` era uma *cópia*),

e é exatamente essa ambiguidade — silenciosa, sem quebrar o programa — que o aviso existe para sinalizar. É um "aliasing" na sua forma mais perigosa: ao contrário do aliasing de listas da Apostila 1, aqui você pode nem saber se o efeito colateral aconteceu ou não sem inspecionar manualmente.

5.3. As duas formas corretas de resolver

Opção 1 — quando você realmente quer um subconjunto independente, copie explicitamente:

```
positivos = df[df["pib"] > 0].copy()   # .copy() garante independência total
positivos["situacao"] = "cresceu"      # seguro: positivos não tem nenhuma relação com df
print(df)                              # df original, sem a coluna "situacao" — garantido
```

Opção 2 — quando você quer, de fato, alterar o DataFrame original, faça filtro e atribuição em **uma única operação** com `.loc`, em vez de dois passos separados:

```
df.loc[df["pib"] > 0, "situacao"] = "cresceu"   # filtra e atribui na mesma chamada, sem
ambiguidade
print(df)
```

```
       pais  pib situacao
0    Brasil  2.2  cresceu
1  Argentina -1.8      NaN
2     Chile  3.1  cresceu
```

A regra prática que evita o problema por completo: **nunca encadeie dois colchetes/seleções separados quando a intenção é atribuir um valor** (`df[mascara]["coluna"] = valor` é sempre suspeito). Se a intenção

é modificar o original, use `df.loc[mascara, "coluna"] = valor` em uma única chamada; se a intenção é trabalhar com uma cópia separada, torne isso explícito com `.copy()`.

5.4. Tabela-resumo do capítulo

Situação	O que fazer
Quero um subconjunto de `df` para analisar sem afetar o original	`sub = df[mascara].copy()`
Quero modificar `df` original com base em um filtro	`df.loc[mascara, "coluna"] = valor` (uma chamada só)
Vi o aviso `SettingWithCopyWarning`	Pare e identifique se há indexação encadeada (`df[...][...] =`); reescreva com `.loc` ou `.copy()`
Preciso passar um subconjunto de `df` para uma função que talvez o modifique	Sempre `.copy()` antes de passar, exatamente como a função pura da Apostila 1 (Capítulo 11.8) evita efeitos colaterais

5.5. Exemplo Resolvido — Corrigindo um pipeline com aviso silencioso

Enunciado: o código abaixo roda sem erro, mas produz o `SettingWithCopyWarning` e o analista suspeita (corretamente) que o resultado final está errado. Identifique o problema e reescreva o trecho de forma segura.

```
df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile", "Peru"],
    "inflacao": [4.5, 98.0, 3.8, 3.2],
})

alta_inflacao = df[df["inflacao"] > 10]
alta_inflacao["alerta"] = True   # gera SettingWithCopyWarning

print(df["alerta"].sum())  # o analista espera 1, mas isso lança KeyError:
                           # 'alerta' nem existe em df!
```

Solução comentada:

O erro de raciocínio do analista: ele *supôs* que `alta_inflacao["alerta"] = True` teria alterado `df` original (adicionando a coluna `"alerta"` também lá), mas isso não é garantido — e, neste caso, de fato não acontece: `df` original nunca ganha a coluna, daí o `KeyError` ao tentar `df["alerta"]`. A correção depende da real intenção:

```
# Se a intenção é marcar isso no DataFrame ORIGINAL:
df["alerta"] = False                                  # cria a coluna com um padrão
df.loc[df["inflacao"] > 10, "alerta"] = True           # atualiza só onde a condição é
verdadeira

print(df["alerta"].sum())  # 1 — agora funciona, sem ambiguidade

# Se a intenção era só analisar o subconjunto, sem alterar df:
alta_inflacao = df[df["inflacao"] > 10].copy()
alta_inflacao["alerta"] = True
print(alta_inflacao)   # subconjunto independente, df original permanece sem a coluna
```

O ponto pedagógico central: o `SettingWithCopyWarning` não é "só um aviso irritante para ignorar" — ele é o pandas admitindo que **não sabe dizer** se sua intenção foi cumprida. Tratá-lo como um erro de fato (parar e reescrever, nunca silenciar com `pd.options.mode.chained_assignment = None`) é a prática correta.

> **Aprofundando a teoria — Copy-on-Write: por que o aviso existe e como o pandas 3.0 o elimina**
> Quando você escreve `df[mascara]["col"] = 0`, são *duas* operações: primeiro `df[mascara]` produz um objeto (que pode ser uma *cópia* ou uma *visão* dos dados originais — o pandas 2.x não garante qual); depois o `["col"] = 0` altera *esse objeto intermediário*, que muitas vezes é descartado. Daí o aviso. A regra de ouro (capítulo 5): faça a seleção e a atribuição numa só chamada, `df.loc[mascara, "col"] = 0`, ou

use `.copy()` explícito. O **Copy-on-Write** (CoW) elimina a ambiguidade: toda seleção se comporta como cópia, e a cópia real é adiada até a primeira modificação. No pandas 2.2 é possível ativá-lo para testar (`mode.copy_on_write`); no 3.0 é o padrão.

```
import warnings
base_cow = pd.DataFrame({"pais": ["Brasil", "Chile", "Peru"], "pib": [2.2, 3.1, 2.5]})

with warnings.catch_warnings(record=True) as avisos:
    warnings.simplefilter("always")
    sub = base_cow[base_cow["pib"] > 2.3]          # pode ser cópia ou visão: o pandas não
promete
    sub["pib"] = 0                            # modifica o objeto intermediário
print("Sem CoW, avisos emitidos:", [a.category.__name__ for a in avisos])
print("Base original após a atribuição encadeada:\n", base_cow.to_string(), sep="")

with pd.option_context("mode.copy_on_write", True):
    base2 = pd.DataFrame({"pais": ["Brasil", "Chile", "Peru"], "pib": [2.2, 3.1, 2.5]})
    sub2 = base2[base2["pib"] > 2.3].copy()
    sub2["pib"] = 0
    base2.loc[base2["pib"] > 2.3, "pib"] = 99      # forma correta, em uma só chamada
print("\nCom CoW e atribuição por .loc:\n", base2.to_string(), sep="")
```

```
Sem CoW, avisos emitidos: ['SettingWithCopyWarning']
Base original após a atribuição encadeada:
     pais  pib
0  Brasil  2.2
1   Chile  3.1
2    Peru  2.5

Com CoW e atribuição por .loc:
     pais   pib
0  Brasil   2.2
1   Chile  99.0
2    Peru  99.0
```

Capítulo 6 — Filtragem booleana

6.1. O que realmente acontece por baixo dos panos

A operação mais usada em pandas é filtrar linhas por condição. A sintaxe:

```
df[df["coluna"] condicao]
```

parece mágica, mas é a composição de dois passos simples:

```
mascara = df["pib"] > 2.0
print(mascara)
```

Saída:

```
0     True
1    False
2     True
```

```
3     True
Name: pib, dtype: bool
```

`df["pib"] > 2.0` não retorna um único `True`/`False` — retorna uma **Series booleana** do mesmo tamanho que `df`, com um `True`/`False` para cada linha, calculada de forma vetorizada (Capítulo 1.2). Em seguida, `df[mascara]` seleciona **apenas as linhas onde a máscara é `True`** — isso é chamado de *boolean indexing* (indexação booleana), e é o mecanismo por trás de todo filtro em pandas:

```
df[mascara]     # equivalente a df[df["pib"] > 2.0]
```

6.2. Combinando condições: por que `and`/`or` não funcionam

```
df[(df["pib"] > 0) & (df["inflacao"] < 10)]   # E lógico
df[(df["pib"] > 3) | (df["pais"] == "Argentina")]  # OU lógico
df[~(df["pais"] == "Brasil")]                  # negação
```

Um erro extremamente comum de quem vem da Apostila 1 é tentar escrever `df["pib"] > 0 and df["inflacao"] < 10`, usando os operadores lógicos `and`/`or`/`not` do Python puro (Capítulo 5.3 da Apostila 1). Isso lança um erro:

```
ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.any() or
a.all().
```

A razão: `and`/`or`/`not` do Python puro são desenhados para operar sobre **um único valor de verdade** por vez (um `bool` escalar). Mas `df["pib"] > 0` é uma **Series inteira** de `True`/`False` — Python não sabe como decidir se uma Series inteira é "verdadeira" ou "falsa" de uma vez só (isso seria ambíguo: bastaria um `True` para considerar a Series toda "verdadeira"? Todos precisariam ser `True`?). Por isso, pandas exige os operadores **bit a bit** `&` (E), `|` (OU) e `~` (negação), que foram redefinidos para funcionar elemento a elemento em arrays/Series inteiras — e, crucialmente, **cada condição precisa estar entre parênteses**, porque `&`/`|` têm precedência mais alta que `>`/`<`/`==` em Python, e sem parênteses a expressão seria avaliada na ordem errada.

	Python puro (escalar)	pandas (elemento a elemento)
E lógico	`and`	`&` (com parênteses em cada lado)
OU lógico	`or`	`|` (com parênteses em cada lado)
Negação	`not`	`~`
Opera sobre	Um `bool` por vez	Uma Series/array inteira, célula a célula

6.3. Métodos de filtro especializados

```
df[df["pais"].isin(["Brasil", "Chile"])]          # está dentro de uma lista de valores
df[df["pib"].between(0, 3)]                        # está entre dois valores (inclusive)
df[df["pais"].str.contains("Bra")]                 # contém um trecho de texto
df[df["pais"].str.startswith("A")]                 # começa com
df[df["pais"].str.endswith("e")]                   # termina com
df[df["pais"].str.contains("bra", case=False)]     # ignora maiúsc./minúsc.
df[df["ipca"].isna()]                              # linhas com dado faltante (Capítulo 8)
```

O acessor `.str` (visto acima em `.str.contains`, `.str.startswith`) expõe, vetorizado sobre a coluna inteira, praticamente todos os métodos de string do Python puro vistos na Apostila 1 (Capítulo 6.4: `.upper()`, `.lower()`, `.strip()`, etc. — todos têm equivalente como `df["coluna"].str.upper()`).

6.4. `.query()` — uma sintaxe alternativa mais legível

```
df.query("pib > 2 and inflacao < 10")        # dentro de query(), and/or funcionam
normalmente!
df.query("pais in ['Brasil', 'Chile']")
df.query("pib > @limite")                     # @variavel referencia uma variável Python
externa
```

Dentro de uma string passada a `.query()`, pandas interpreta a expressão com seu próprio motor (baseado na biblioteca `numexpr`, quando disponível), que entende `and`/`or`/`in` da forma "natural" — porque, nesse contexto, não há ambiguidade sobre Series (a expressão inteira já é avaliada linha a linha pelo motor de `query`). É uma questão de gosto e legibilidade: para filtros complexos com muitas condições, `.query()` costuma ser mais fácil de ler do que uma cadeia longa de `&`/`|` com parênteses.

6.5. Exemplo Resolvido — Diagnosticando um filtro que retorna vazio

Enunciado: o código abaixo deveria retornar os países com PIB positivo **e** inflação controlada (abaixo de 10%), mas retorna um DataFrame vazio. Identifique os dois erros.

```
df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile", "Colômbia"],
    "pib": [2.2, -1.8, 3.1, 2.8],
    "inflacao": [4.5, 98.0, 3.8, 7.2],
})

resultado = df[df["pib"] > 0 and df["inflacao"] < 10]
```

Solução comentada:

Há dois problemas na linha:

1. **`and` em vez de `&`** — como visto na Seção 6.2, isso deveria lançar um `ValueError` ("truth value of a Series is ambiguous"), não silenciosamente devolver algo. Ou seja, o primeiro problema real é que esse código **nem chegaria a rodar** — o enunciado descreve um sintoma que só ocorreria se o erro fosse outro; isso reforça o hábito de sempre ler a mensagem de erro completa (Apostila 1, Capítulo 13.1) em vez de assumir qual foi a causa.
2. **Falta de parênteses** em cada condição — mesmo trocando `and` por `&`, `df["pib"] > 0 & df["inflacao"] < 10` seria avaliado incorretamente, porque `&` tem precedência maior que `>`, então o Python tentaria calcular `0 & df["inflacao"]` antes de comparar com `df["pib"]` — um erro de precedência de operadores.

A correção:

```
resultado = df[(df["pib"] > 0) & (df["inflacao"] < 10)]
print(resultado)
```

```
     pais  pib  inflacao
0  Brasil  2.2       4.5
2   Chile  3.1       3.8
3  Colômbia 2.8       7.2
```

A forma equivalente com `.query()`, que evita os dois problemas de uma vez (usa `and` normalmente e não exige parênteses adicionais):

```
resultado = df.query("pib > 0 and inflacao < 10")
```

Exemplo resolvido — filtros úteis: `isin`, `between`, `str.contains`, `~` e `query` com variáveis
Sobre a abertura do IPCA (acumulado em 12 meses). Cada filtro devolve uma **máscara booleana** combinam-se com &, | e ~ (negação), sempre entre parênteses:

```
meta = 4.5
acima_teto = grupos[grupos["IPCA cheio"] > meta]
servicos_acima = grupos.query("Servicos > `IPCA cheio`")                # crase para
colunas com espaço
faixa = grupos[grupos["IPCA cheio"].between(3.0, 4.5)]
print(f"Meses com IPCA 12m acima do teto ({meta}%): {len(acima_teto)} de {len(grupos)}")
print(f"Serviços acima do IPCA cheio: {len(servicos_acima)} meses | IPCA entre a meta e o
teto: {len(faixa)} meses")
inflacao_serv_e_livres = grupos[(grupos["Servicos"] > 5) & ~(grupos["Administrados"] < 0)]
print("Serviços > 5% e administrados NÃO negativos:", len(inflacao_serv_e_livres), "meses")

nomes = pd.Series(["Banco Alfa", "Cia Beta", "Banco Gama", None])
print("str.contains('Banco'):", nomes.str.contains("Banco", na=False).tolist(), "| isin:",
nomes.isin(["Cia Beta"]).tolist())
limite = 5.0                                                            # variável externa
na query com @
print("query com @limite:", len(grupos.query("Servicos > @limite")), "meses")
```

```
Meses com IPCA 12m acima do teto (4.5%): 24 de 43
Serviços acima do IPCA cheio: 40 meses | IPCA entre a meta e o teto: 19 meses
Serviços > 5% e administrados NÃO negativos: 30 meses
str.contains('Banco'): [True, False, True, False] | isin: [False, True, False, False]
query com @limite: 36 meses
```

Atenção — erro comum
`NaN` não é igual a nada, nem a si mesmo. `df["x"] == np.nan` devolve sempre `False`: use `isna()`. E `and`/`or` em máscaras provocam o erro de "valor de verdade ambíguo": use &, | com parênteses, porque o operador & tem *precedência maior* que >.

Capítulo 7 — Criando, transformando e removendo colunas

7.1. Atribuição direta vetorizada (a forma preferida)

```
df["pib_per_capita"] = df["pib"] / df["populacao"] * 1000
df["log_pib"] = np.log(df["pib"] + 10)
df["inflacao_decimal"] = df["inflacao"] / 100   # atualiza coluna existente
```

Cada uma dessas linhas é vetorizada (Capítulo 1.2): a divisão, multiplicação e o logaritmo (`np.log`, do numpy) são aplicados à coluna **inteira** de uma vez, em código compilado, sem nenhum laço Python explícito. Essa é, de longe, a forma mais rápida e mais idiomática de criar uma coluna em pandas.

7.2. `.apply()` com função ou `lambda`

Quando a transformação não é uma operação aritmética simples entre colunas — por exemplo, uma regra condicional linha a linha —, `.apply()` permite rodar uma função Python arbitrária sobre cada elemento (ou cada linha):

```
# lambda inline
df["classificacao"] = df["pib"].apply(lambda x: "Crescendo" if x > 0 else "Recessão")
```

```python
# função nomeada, para regras mais elaboradas (mais legível, testável isoladamente)
def classificar_pib(x):
    if x > 3:
        return "Alto"
    elif x > 0:
        return "Médio"
    else:
        return "Negativo"

df["categoria"] = df["pib"].apply(classificar_pib)

# apply em uma LINHA inteira (axis=1) — acessa várias colunas ao mesmo tempo
df["situacao"] = df.apply(
    lambda linha: "Estagflação" if linha["pib"] < 0 and linha["inflacao"] > 5 else
"Normal",
    axis=1,
)
```

7.3. Por que `.apply()` é mais lento — e quando isso importa

Este é o análogo direto, em pandas, da comparação "laço vs. vetorização" vista na Apostila 1 (Capítulo 10.4). `.apply()` **parece** vetorizado (é uma única linha, aplicada à coluna "inteira"), mas por baixo dos panos, para a maioria dos casos, pandas ainda executa a função Python **uma vez por linha**, em um laço interno — perdendo o benefício de delegar tudo ao numpy em C. A diferença de desempenho é irrelevante em tabelas de algumas centenas de linhas, mas se torna significativa (pode ser 10 a 100 vezes mais lento) em tabelas com milhões de linhas, como microdados da PNAD ou séries diárias de alta frequência.

Sempre que existir uma alternativa vetorizada, prefira-a:

```python
# Mais lento (apply percorre linha a linha em Python)
df["classificacao"] = df["pib"].apply(lambda x: "Alto" if x > 3 else ("Médio" if x > 0 else
"Negativo"))

# Mais rápido (np.select é vetorizado: avalia todas as condições de uma vez, em C)
condicoes = [df["pib"] > 3, df["pib"] > 0]
categorias = ["Alto", "Médio"]
df["classificacao"] = np.select(condicoes, categorias, default="Negativo")

# Também vetorizado, para uma única condição binária
df["cresceu"] = np.where(df["pib"] > 0, "Sim", "Não")
```

`np.where(condicao, valor_se_verdadeiro, valor_se_falso)` é o equivalente vetorizado do operador ternário do Python (Apostila 1, Capítulo 9.3), aplicado a uma coluna inteira de uma vez. `np.select` generaliza isso para múltiplas condições em cascata, substituindo uma cadeia de `if/elif/else` (Apostila 1, Capítulo 9.1) por uma única chamada vetorizada.

Ferramenta	Quando usar	Velocidade
Operação aritmética direta (`df["a"] + df["b"]`)	Sempre que a transformação é matemática entre colunas	Mais rápida (100% vetorizada em C)
`np.where`	Uma condição binária (se/senão)	Rápida (vetorizada)
`np.select`	Várias condições em cascata (equivalente a `if/elif/elif/else`)	Rápida (vetorizada)
`.map()` (com dicionário)	Substituir valores de acordo com um mapeamento fixo	Rápida
`.apply()`	Lógica arbitrária, complexa demais para as opções acima	Mais lenta — último recurso

7.4. `.map()`, `.replace()` e renomeação/remoção de colunas

```
# .map() com um dicionário — substituição elemento a elemento
sigla_para_nome = {"BRA": "Brasil", "ARG": "Argentina", "CHL": "Chile"}
df["nome_pais"] = df["sigla"].map(sigla_para_nome)

# .replace() — troca valores específicos, sem exigir mapear TODOS os valores
df["pais"] = df["pais"].replace({"Bra": "Brasil", "Arg": "Argentina"})

# Renomear colunas
df = df.rename(columns={"pib": "crescimento_pib"})
df.rename(columns={"pib": "crescimento_pib"}, inplace=True)  # altera no lugar, sem
reatribuir

# Remover coluna(s)
df = df.drop("coluna_que_nao_quero", axis=1)
df = df.drop(columns=["col_a", "col_b"])   # forma mais explícita, sem precisar de axis=1

# Reordenar colunas (basta selecionar na ordem desejada)
df = df[["pais", "pib", "inflacao", "populacao"]]
```

Nota sobre `inplace=True`: esse parâmetro modifica o DataFrame no próprio objeto, sem exigir reatribuição (`df.rename(..., inplace=True)` em vez de `df = df.rename(...)`). Parece conveniente, mas tem o mesmo tipo de risco de ambiguidade visto no Capítulo 5: se `df` for, ele mesmo, uma *view* de outro DataFrame, `inplace=True` pode disparar o mesmo `SettingWithCopyWarning`. A comunidade pandas vem, gradualmente, recomendando **evitar `inplace=True`** e preferir sempre a forma que retorna um novo objeto (`df = df.rename(...)`), por ser mais previsível e permitir encadeamento de métodos (*method chaining*, Capítulo 14).

7.5. Exemplo Resolvido — Reescrevendo um `.apply()` como vetorizado

Enunciado: o código abaixo classifica o regime de juro real (Apostila 1, Capítulo 5.4) usando `.apply()`. Reescreva-o de forma totalmente vetorizada, sem nenhuma função Python chamada linha a linha.

```
df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile", "Turquia"],
    "selic": [10.5, 80.0, 5.5, 45.0],
    "ipca": [4.2, 120.0, 3.8, 55.0],
})

def classifica_juro_real(linha):
    juro_real = linha["selic"] - linha["ipca"]
    if juro_real > 3:
        return "Fortemente positivo"
    elif juro_real > 0:
        return "Positivo"
    else:
        return "Negativo"

df["regime"] = df.apply(classifica_juro_real, axis=1)
```

Solução comentada:

```
# Passo 1: a subtração já é vetorizada (não precisa de apply para isso)
df["juro_real"] = df["selic"] - df["ipca"]

# Passo 2: np.select troca o if/elif/else linha a linha por uma
# avaliação vetorizada das condições sobre a coluna inteira
condicoes = [df["juro_real"] > 3, df["juro_real"] > 0]
categorias = ["Fortemente positivo", "Positivo"]
df["regime"] = np.select(condicoes, categorias, default="Negativo")
```

```
print(df[["pais", "juro_real", "regime"]])
```

```
       pais  juro_real              regime
0    Brasil        6.3  Fortemente positivo
1 Argentina      -40.0             Negativo
2     Chile        1.7             Positivo
3   Turquia      -10.0             Negativo
```

O ganho de reescrever não é visível em 4 linhas — é visível quando `df` tem 5 milhões de linhas (como uma base de crédito do SCR do BCB, por exemplo): `.apply(..., axis=1)` chamaria a função Python `classifica_juro_real` 5 milhões de vezes, uma por uma; a versão com `np.select` avalia as duas condições sobre o array inteiro de uma vez, delegando o trabalho ao numpy. A lição estrutural: sempre que uma transformação "linha a linha" pode ser reescrita como uma sequência de operações entre colunas inteiras (subtração + comparação), prefira essa forma.

Exemplo resolvido — classificar em faixas: `pd.cut`, `rank`, `clip` e `where`
O IPCA acumulado em 12 meses, classificado em relação ao regime de metas (meta de 3% com tolerância de 1,5 ponto percentual para cima e para baixo — o intervalo de 1,5% a 4,5%):

```
faixas = pd.cut(ipca12, bins=[-np.inf, 1.5, 3.0, 4.5, np.inf],
                labels=["abaixo do piso", "entre piso e meta", "entre meta e teto", "acima
do teto"])
print(faixas.value_counts(sort=False).to_string())
print("\nRanking (1 = IPCA mais alto) dos 3 maiores:",
ipca12.rank(ascending=False).nsmallest(3).index.strftime("%m/%Y").tolist())
print("Limitado entre 3 e 5 (clip) - min/max:", ipca12.clip(3, 5).min(), ipca12.clip(3,
5).max())
print("where (valor ou 'NaN' fora da banda):", ipca12.where(ipca12.between(1.5,
4.5)).isna().sum(), "meses fora da banda")

texto = pd.Series(["2024-Q1: 3.2%", "2024-Q2: 2.9%", "2024-Q3: n/d"])
print(texto.str.extract(r"(\d{4})-Q(\d): ([\d.]+)%").rename(columns={0: "ano", 1: "tri", 2:
"valor"}).to_string())
```

```
ipca_12m
abaixo do piso        0
entre piso e meta     0
entre meta e teto    19
acima do teto        24

Ranking (1 = IPCA mais alto) dos 3 maiores: ['01/2023', '02/2023', '04/2025']
Limitado entre 3 e 5 (clip) - min/max: 3.16 5.0
where (valor ou 'NaN' fora da banda): 24 meses fora da banda
    ano  tri valor
0  2024    1   3.2
1  2024    2   2.9
2   NaN  NaN   NaN
```

Capítulo 8 — Dados faltantes (`NaN`) em profundidade

8.1. O que é `NaN`, tecnicamente

`NaN` significa *Not a Number* — não é uma invenção do pandas, é um valor especial definido no padrão **IEEE 754**, o mesmo padrão que rege a representação de números de ponto flutuante em praticamente qualquer linguagem

(o mesmo padrão citado na Apostila 1, Capítulo 4.2, a propósito de `0.1 + 0.2`). `NaN` é tecnicamente um `float`, e isso tem uma consequência direta e importante:

> **Uma coluna `int64` que ganha um valor faltante é automaticamente promovida a `float64`** — porque não existe uma representação de `NaN` dentro do espaço de bits reservado para inteiros. Uma coluna de população (`int64`) que, ao ser cruzada com outra fonte, ganha um `NaN` em uma linha, silenciosamente vira `float64` inteira, e valores que eram `214` passam a aparecer como `214.0`.

```
populacao = pd.Series([214, 46, 19], dtype="int64")
populacao_com_falha = pd.Series([214, None, 19])
print(populacao_com_falha.dtype)   # float64, não int64!
print(populacao_com_falha)
# 0    214.0
# 1      NaN
# 2     19.0
# dtype: float64
```

Isso é diferente de `None`, o valor "vazio" do Python puro (Apostila 1, Capítulo 9.5): `None` é um objeto Python genérico, que pode aparecer em qualquer coluna `object`; `NaN` é especificamente um valor de ponto flutuante. Quando você escreve `None` dentro de uma lista/Series que pandas está tentando tornar numérica, pandas geralmente o **converte para `NaN`** automaticamente, como no exemplo acima.

Desde a versão 1.0, pandas introduziu **`pd.NA`**, um marcador de ausência mais genérico, pensado para funcionar com os **dtypes anuláveis** (`Int64`, `boolean`, `string` — com letra maiúscula, diferentes de `int64`, `bool`, `object`). Uma coluna `Int64` (nullable) pode ter um valor faltante **sem** perder o tipo inteiro:

```
populacao_nullable = pd.Series([214, None, 19], dtype="Int64")  # "I" maiúsculo
print(populacao_nullable)
# 0     214
# 1    <NA>
# 2      19
# dtype: Int64
```

Uma propriedade de `NaN`, herdada diretamente do padrão IEEE 754, explica por que **nunca** se testa dado faltante com `==`:

```
import numpy as np
print(np.nan == np.nan)   # False!
```

Por definição do padrão, `NaN` não é igual a **nada**, nem a si mesmo — é o único valor de ponto flutuante com essa propriedade (reflete a ideia de que "não sei o valor" comparado a "não sei o valor" não deveria ser afirmado como igual). Na prática, isso significa que `df[df["coluna"] == np.nan]` **nunca** encontra as linhas faltantes — sempre retorna vazio, silenciosamente, sem erro. A forma correta é sempre `.isnull()`/`.isna()` (Seção 8.2), que testam o *tipo* do valor, não sua igualdade.

8.2. Detectando dados faltantes

```
df.isnull()           # DataFrame de True/False, célula a célula (isna() é sinônimo exato)
df.isnull().sum()      # quantos nulos por coluna
df.isnull().sum().sum()  # total de nulos na tabela inteira
df.notnull()           # o oposto de isnull()

df["ipca"].isna()      # versão para uma única coluna
```

8.3. Removendo linhas/colunas com dado faltante

```
df.dropna()                          # remove QUALQUER linha com pelo menos um nulo
df.dropna(subset=["pib"])            # remove só se "pib" especificamente for nulo
df.dropna(how="all")                 # remove só se TODOS os valores da linha forem nulos
df.dropna(thresh=3)                  # mantém a linha se tiver ao menos 3 valores não-nulos
df.dropna(axis=1)                    # remove COLUNAS (não linhas) com algum nulo
```

8.4. Preenchendo dados faltantes

```
df.fillna(0)                          # preenche com um valor fixo
df.fillna(df.mean(numeric_only=True))  # preenche cada coluna com sua própria média
df["pib"].fillna(df["pib"].median()) # preenche com a mediana (mais robusta a outliers que
a média)

df["ipca"].ffill()   # "forward fill" — repete o último valor válido conhecido
df["ipca"].bfill()   # "backward fill" — usa o próximo valor válido conhecido

df["ipca"].interpolate()   # interpolação linear entre os valores vizinhos válidos
```

A escolha entre essas estratégias depende do **motivo** do dado faltar — um ponto conceitual, não apenas técnico, que muitos tutoriais pulam:

Situação	Estratégia recomendada	Por quê
Feriado bancário (câmbio, SELIC diária)	`ffill()`	O valor do dia útil anterior é a melhor aproximação — o mercado só não operou
Divulgação atrasada do IBGE (dado ainda não saiu)	Deixar como `NaN`, não preencher	Preencher "inventaria" um dado que ainda não existe; melhor esperar ou marcar explicitamente como projeção
Erro de digitação isolado, sem padrão	`interpolate()` ou remover a linha	Depende do quanto esse ponto isolado afeta a análise (ex: uma média móvel)
Série realmente descontínua (país começou a existir em certo ano)	`dropna()` no período sem dado	Preencher artificialmente distorceria qualquer estatística sobre o período anterior

Armadilha comum: preencher `NaN` com `0` em uma coluna de taxa de crescimento (PIB, inflação) é quase sempre um erro — `0` tem um significado econômico próprio ("estabilidade", "nem cresceu nem caiu"), e não deve ser confundido com "não sei o valor". Preencher com `0` silenciosamente distorce qualquer média ou correlação calculada depois.

8.5. Exemplo Resolvido — Diagnosticando a causa de um `NaN` antes de tratá-lo

Enunciado: ao rodar `df.isnull().sum()` você obtém:

```
data        0
selic       0
ipca        2
```

```
cambio     45
dtype: int64
```

A coluna `cambio` (câmbio diário) tem 45 valores faltantes em uma série de 260 dias úteis de um ano; a coluna `ipca` (mensal) tem 2 valores faltantes em 12 meses. Antes de escolher `ffill`, `dropna` ou `interpolate`, que hipótese você investigaria para cada coluna, e como confirmaria cada hipótese em código?

Solução comentada:

Para `cambio`: 45 em 260 dias úteis é compatível com feriados nacionais e municipais (não todo feriado é feriado bancário em todo lugar, mas a ordem de grandeza bate). A hipótese: os `NaN` se concentram em datas específicas conhecidas (feriados), não espalhados aleatoriamente.

```
print(df[df["cambio"].isna()]["data"])   # inspeciona quais datas específicas faltam
```

Se as datas baterem com feriados conhecidos, `ffill()` é apropriado (o câmbio do último dia útil é a melhor aproximação para um dia sem pregão):

```
df["cambio"] = df["cambio"].ffill()
```

Para `ipca`: 2 em 12 meses é uma proporção muito mais alta (16%) para uma série mensal que deveria ser completa — isso sugere um problema de importação (Capítulo 2.6/3.4), não uma ausência genuína de dado. A hipótese aqui é diferente: **provavelmente não é dado realmente ausente, é erro de leitura**.

```
print(df[df["ipca"].isna()])   # inspeciona as linhas inteiras — o valor bruto (antes do
parse) ajuda
```

Se a inspeção mostrar que essas duas linhas tinham um valor como `"-"` ou vírgula decimal mal interpretada, a correção correta é **voltar ao Capítulo 2** e ajustar `na_values`/`decimal` na importação — não aplicar `ffill()` cegamente sobre um problema que na verdade é de leitura do arquivo. A lição central: **o tratamento certo de um `NaN` depende de diagnosticar a causa primeiro**; aplicar `fillna()` ou `dropna()` sem entender por que o dado falta é tratar o sintoma, não a causa.

> **Aprofundando a teoria — por que os dados faltam: MCAR, MAR e MNAR**
> O tratamento certo depende de *por que* o dado está faltando:
>
> - **MCAR** (missing completely at random): a ausência não depende de nada (falha de sensor). Descartar linhas ou imputar a média quase não enviesa.
> - **MAR** (missing at random): depende de outra variável *observada* (pessoas jovens respondem menos a um item). Imputação condicional (por grupo) funciona.
> - **MNAR** (missing not at random): depende do próprio valor ausente (quem tem renda alta omite a renda). Nenhuma técnica simples resolve; o viés é inevitável e deve ser reportado.
>
> Em séries temporais há uma pergunta extra: a série é um **fluxo suave** (câmbio, onde interpolar faz sentido) ou um **degrau** (meta da Selic, que muda em reuniões, e onde preencher com o último valor é o certo)?

```
rng = np.random.default_rng(3)
verdadeira = selic_d.loc["2025-01-01":"2025-12-31"]
com_buracos = verdadeira.copy()
com_buracos[rng.random(len(com_buracos)) < 0.2] = np.nan                    # 20% de
buracos aleatórios (MCAR)
metodos = {
    "ffill (último valor)": com_buracos.ffill(),
    "interpolate (linear)": com_buracos.interpolate(),
    "dropna (descartar)": com_buracos.dropna(),
}
```

```python
print(f"Buracos: {com_buracos.isna().sum()} de {len(com_buracos)} | média verdadeira:
{verdadeira.mean():.4f}")
for nome_m, serie_m in metodos.items():
    erro = (serie_m.reindex(verdadeira.index) - verdadeira).abs().mean()
    print(f"{nome_m:<22} média = {serie_m.mean():.4f} | erro absoluto médio por dia =
{erro:.4f}")
```

```
Buracos: 64 de 365 | média verdadeira: 14.4171
ffill (último valor)   média = 14.4231 | erro absoluto médio por dia = 0.0000
interpolate (linear)   média = 14.4272 | erro absoluto médio por dia = 0.0041
dropna (descartar)     média = 14.4244 | erro absoluto médio por dia = 0.0000
```

Intuição
A Selic meta é um **degrau**: o último valor conhecido é, quase sempre, o valor verdadeiro, e `ffill` acerta. A interpolação linear "inventa" valores intermediários entre dois degraus — um erro pequeno, mas que não existe na realidade. Conhecer o *processo gerador* do dado é o que decide a técnica; o código só a executa.

Capítulo 9 — Agrupamento (`groupby`)

9.1. O paradigma split-apply-combine

`groupby` é o equivalente de uma tabela dinâmica do Excel, mas a lógica por trás dele tem nome formal, cunhado por Hadley Wickham (autor do pacote `dplyr`, de R) e adotado também pela documentação de pandas: **split--apply-combine**.

```
            SPLIT                  APPLY                 COMBINE
      (divide por grupo)     (aplica uma função      (junta os resultados
                               a cada grupo)            de volta em uma tabela)

  df.groupby("pais")  →   ["pib"].mean()  →   uma linha por país, com a média
```

1. **Split**: os dados são divididos em grupos, de acordo com os valores de uma ou mais colunas (`groupby("pais")` cria um grupo para cada país distinto).
2. **Apply**: uma função (média, soma, contagem, ou uma função personalizada) é aplicada **independentemente** a cada grupo.
3. **Combine**: os resultados de cada grupo são recombinados em uma única estrutura de saída.

```python
dados = {
    "ano": [2020, 2020, 2020, 2021, 2021, 2021],
    "pais": ["Brasil", "Argentina", "Chile", "Brasil", "Argentina", "Chile"],
    "pib": [-3.3, -9.9, -6.1, 4.8, 10.4, 11.7],
}
df = pd.DataFrame(dados)

df.groupby("pais")["pib"].mean()
```

```
pais
Argentina    0.25
Brasil       0.75
Chile        2.80
Name: pib, dtype: float64
```

Repare que `df.groupby("pais")` sozinho **não retorna uma tabela** — retorna um objeto intermediário (`DataFrameGroupBy`), que representa a "divisão" ainda sem a etapa de aplicar/combinar. É só ao encadear `["pib"].mean()` que as três etapas se completam.

9.2. Múltiplas agregações com `.agg()`

```
df.groupby("pais")["pib"].agg(["mean", "min", "max", "std"])

# Nomeando as colunas de saída explicitamente (agregação nomeada,
# a forma mais legível e recomendada em pandas moderno)
df.groupby("pais")["pib"].agg(
    media_pib=("pib", "mean"),
    minimo_pib=("pib", "min"),
    volatilidade=("pib", "std"),
)

# Agregações diferentes para colunas diferentes
df.groupby("pais").agg({
    "pib": ["mean", "std"],
    "ano": "count",
})
```

9.3. Agrupando por múltiplas colunas

```
df.groupby(["ano", "pais"])["pib"].sum()
```

```
ano   pais
2020  Argentina    -9.9
      Brasil       -3.3
      Chile        -6.1
2021  Argentina    10.4
      Brasil        4.8
      Chile        11.7
Name: pib, dtype: float64
```

O resultado tem um **MultiIndex** (índice hierárquico, com dois níveis: ano e país) — a extensão natural do Index visto no Capítulo 1.4 para mais de uma dimensão, análoga ao dicionário aninhado da Apostila 1 (Capítulo 8.4: `pib_por_ano["Brasil"]["2024"]`), só que como estrutura tabular nativa do pandas. Para voltar a um formato "plano" (colunas normais, sem hierarquia), use `.reset_index()`:

```
resultado = df.groupby(["ano", "pais"])["pib"].sum().reset_index()
```

9.4. `.transform()` vs. `.agg()` — resultado do mesmo tamanho ou reduzido

Uma distinção que confunde iniciantes: `.agg()` (e `.mean()`, `.sum()`, etc.) **reduz** cada grupo a um único valor — a tabela de saída é menor que a de entrada (uma linha por grupo). `.transform()`, por outro lado, devolve um resultado do **mesmo tamanho** da tabela original, repetindo o valor agregado em cada linha do seu grupo — útil para comparar cada observação individual contra a média do seu próprio grupo:

```
# Cria uma coluna com a média do PIB do PRÓPRIO grupo (país), repetida em cada linha
df["pib_medio_pais"] = df.groupby("pais")["pib"].transform("mean")

# Isso permite calcular, por exemplo, o desvio de cada observação frente à média do seu
país
df["desvio_da_media_do_pais"] = df["pib"] - df["pib_medio_pais"]

print(df)
```

```
    ano       pais   pib  pib_medio_pais  desvio_da_media_do_pais
0  2020     Brasil  -3.3            0.75                    -4.05
1  2020  Argentina  -9.9            0.25                   -10.15
2  2020      Chile  -6.1            2.80                    -8.90
3  2021     Brasil   4.8            0.75                     4.05
4  2021  Argentina  10.4            0.25                    10.15
5  2021      Chile  11.7            2.80                     8.90
```

9.5. Exemplo Resolvido — Volatilidade do PIB por país, ordenada

Enunciado: dado o `df` com PIB de vários países ao longo de vários anos (mesma estrutura da Seção 9.1, mas com mais anos), calcule o desvio-padrão do PIB de cada país e ordene do mais volátil ao mais estável. Em seguida, identifique quais países têm volatilidade acima da média de todos os países.

```
dados = {
    "ano": [2019, 2020, 2021, 2019, 2020, 2021, 2019, 2020, 2021],
    "pais": ["Brasil", "Brasil", "Brasil", "Argentina", "Argentina", "Argentina",
             "Chile", "Chile", "Chile"],
    "pib": [1.2, -3.3, 4.8, -2.0, -9.9, 10.4, 0.8, -6.1, 11.7],
}
df = pd.DataFrame(dados)
```

Solução comentada:

```
volatilidade = df.groupby("pais")["pib"].std().sort_values(ascending=False)
print(volatilidade)
```

```
pais
Argentina    10.232790
Chile         8.974594
Brasil        4.058325
Name: pib, dtype: float64
```

(A ordem exata depende do cálculo — o ponto é o método, não os números decorados.)

```
media_das_volatilidades = volatilidade.mean()
acima_da_media = volatilidade[volatilidade > media_das_volatilidades]
print(acima_da_media)
```

O desvio-padrão amostral que `.std()` calcula segue:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Note o `n-1` no denominador (correção de Bessel) — voltaremos a essa fórmula, e a uma armadilha importante envolvendo ela, no Capítulo 12.1. O raciocínio de todo o exercício segue o padrão split-apply-combine: **split** por país, **apply** do desvio-padrão a cada grupo, **combine** em uma Series indexada por país — e, depois, uma segunda agregação (a média das volatilidades) sobre o resultado já agregado, um padrão comum em análises de "agregação de agregações" (ex: volatilidade média entre países de uma região).

Exemplo resolvido — agrupar por período: `Grouper`, `size` × `count`, `crosstab` e ranking dentro do grupo
Sobre a Selic diária (mudando a *base* do agrupamento de país para *tempo*):

```
por_trimestre = selic_d.groupby(pd.Grouper(freq="QE")).agg(media="mean", minimo="min",
maximo="max", dias="size")
print(por_trimestre.tail(4).round(2).to_string())
```

```
mudancas_ano = (selic_d.diff().fillna(0) != 0).groupby(selic_d.index.year).sum()      #
conta mudanças de meta por ano
print("Mudanças da meta por ano:", mudancas_ano.to_dict())

df_g = pd.DataFrame({"setor": ["Energia", "Energia", "Varejo", "Varejo", "Varejo"],
                     "uf": ["CE", "SP", "CE", "SP", "SP"],
                     "receita": [10.0, 12.0, 8.0, np.nan, 9.0]})
print("\nsize (inclui NaN):", df_g.groupby("setor").size().to_dict(), "| count (ignora
NaN):", df_g.groupby("setor")["receita"].count().to_dict())
print(pd.crosstab(df_g["setor"], df_g["uf"]).to_string())
df_g["rank_no_setor"] = df_g.groupby("setor")["receita"].rank(ascending=False)
print(df_g.to_string())
```

```
            media  minimo  maximo  dias
data
2025-12-31  15.00   15.00   15.00    92
2026-03-31  14.96   14.75   15.00    90
2026-06-30  14.54   14.25   14.75    91
2026-09-30  14.12   14.00   14.25    78
Mudanças da meta por ano: {2023: 4, 2024: 6, 2025: 4, 2026: 4}

size (inclui NaN): {'Energia': 2, 'Varejo': 3} | count (ignora NaN): {'Energia': 2,
'Varejo': 2}
uf       CE  SP
setor
Energia   1   1
Varejo    1   2
     setor  uf  receita  rank_no_setor
0  Energia  CE     10.0            2.0
1  Energia  SP     12.0            1.0
2   Varejo  CE      8.0            2.0
3   Varejo  SP      NaN            NaN
4   Varejo  SP      9.0            1.0
```

Capítulo 10 — Combinando tabelas: `merge`, `concat`, `join`

10.1. `merge` — juntando por uma chave comum

`pd.merge` é o equivalente ao PROCV/PROCX do Excel, ou ao `JOIN` de SQL — mas opera em ambas as tabelas simultaneamente, e não célula por célula.

```
pib_df = pd.DataFrame({"pais": ["Brasil", "Argentina", "Chile"], "pib": [2.2, -1.8, 3.1]})
inf_df = pd.DataFrame({"pais": ["Brasil", "Argentina", "Peru"], "inflacao": [4.5, 98.0,
3.2]})

# INNER (padrão): só as linhas cuja chave existe nas DUAS tabelas
pd.merge(pib_df, inf_df, on="pais")
```

```
       pais  pib  inflacao
0    Brasil  2.2       4.5
1  Argentina -1.8      98.0
```

Repare que `Chile` (só está em `pib_df`) e `Peru` (só está em `inf_df`) desaparecem — essa é a característica do `inner join`: só sobrevive a interseção.

```python
# LEFT: todas as linhas da tabela ESQUERDA, mesmo sem par na direita (vira NaN)
pd.merge(pib_df, inf_df, on="pais", how="left")

# RIGHT: todas as linhas da tabela DIREITA
pd.merge(pib_df, inf_df, on="pais", how="right")

# OUTER: união de tudo, com NaN onde faltar
pd.merge(pib_df, inf_df, on="pais", how="outer")
```

how=	Mantém	Analogia visual (conjuntos)
`"inner"` (padrão)	Só chaves presentes nas **duas** tabelas	Interseção
`"left"`	Todas as chaves da tabela à **esquerda**, completando com `NaN` o que faltar na direita	Tudo do círculo esquerdo
`"right"`	Todas as chaves da tabela à **direita**	Tudo do círculo direito
`"outer"`	União — todas as chaves de ambas, `NaN` onde não houver correspondência	União completa

```python
# Quando os nomes das colunas-chave são diferentes entre as tabelas
pd.merge(pib_df, inf_df, left_on="pais", right_on="country")

# indicator=True adiciona uma coluna "_merge" dizendo a origem de cada linha
# ("left_only", "right_only", "both") -- ótimo para auditar merges
resultado = pd.merge(pib_df, inf_df, on="pais", how="outer", indicator=True)
print(resultado["_merge"].value_counts())
```

Armadilha comum: merge "muitos-para-muitos" acidental. Se a coluna-chave tiver valores duplicados nas duas tabelas (por exemplo, várias linhas de "Brasil" em cada uma, por engano de importação), `merge` gera o **produto cartesiano** das duplicatas — silenciosamente multiplicando o número de linhas, sem erro. O parâmetro `validate` protege contra isso:

```python
pd.merge(pib_df, inf_df, on="pais", validate="one_to_one")
# lança um erro explícito (MergeError) se qualquer um dos dois lados tiver chaves
duplicadas
```

10.2. `concat` — empilhando tabelas

`concat` serve a um propósito diferente de `merge`: **não junta por chave**, apenas **empilha** tabelas, seja adicionando linhas (mais comum: juntar o mesmo tipo de dado de anos ou fontes diferentes) ou colunas.

```python
pib_2023 = pd.DataFrame({"pais": ["Brasil", "Chile"], "pib": [2.9, 0.2]})
pib_2024 = pd.DataFrame({"pais": ["Brasil", "Chile"], "pib": [3.1, 2.6]})

# Empilha linhas (axis=0, o padrão) — como um UNION em SQL
todos_anos = pd.concat([pib_2023, pib_2024], keys=["2023", "2024"])
```

```
# ignore_index=True descarta os índices originais e cria um novo RangeIndex sequencial
todos_anos = pd.concat([pib_2023, pib_2024], ignore_index=True)

# Empilha colunas lado a lado (axis=1) -- exige que os índices estejam alinhados
lado_a_lado = pd.concat([pib_df.set_index("pais"), inf_df.set_index("pais")], axis=1)
```

10.3. `join` — atalho baseado no Index

```
pib_indexado = pib_df.set_index("pais")
inf_indexada = inf_df.set_index("pais")

resultado = pib_indexado.join(inf_indexada, how="left")
```

`.join()` é essencialmente um `merge` especializado que usa o **Index** de ambas as tabelas como chave, em vez de uma coluna explícita — conveniente quando os dados já estão indexados pela mesma coisa (o que reforça, de novo, por que o Capítulo 1.4 tratou o Index como um cidadão de primeira classe, não como um detalhe cosmético).

10.4. Exemplo Resolvido — Cruzando três fontes com chaves diferentes

Enunciado: você tem PIB (BCB), inflação (IBGE, via Ipeadata) e câmbio médio anual (BCB) de países da América Latina, mas cada fonte usa uma convenção de nome de país ligeiramente diferente, e nem todas cobrem os mesmos países. Junte as três em uma única tabela, preservando todos os países que aparecem em **pelo menos uma** fonte, e identifique quais países têm dado incompleto.

```
pib = pd.DataFrame({"pais": ["Brasil", "Argentina", "Chile", "Peru"], "pib": [2.2, -1.8,
3.1, 2.5]})
inflacao = pd.DataFrame({"country": ["Brasil", "Argentina", "Colômbia"], "inflacao": [4.5,
98.0, 7.2]})
cambio = pd.DataFrame({"pais": ["Brasil", "Chile", "Colômbia"], "cambio_medio": [5.45,
950.2, 4100.0]})
```

Solução comentada:

```
# Passo 1: merge outer entre pib e inflacao, com nomes de coluna-chave diferentes
etapa1 = pd.merge(pib, inflacao, left_on="pais", right_on="country", how="outer")
etapa1 = etapa1.drop(columns="country")   # "country" é redundante após o merge

# Passo 2: merge outer do resultado com cambio
completo = pd.merge(etapa1, cambio, on="pais", how="outer")
print(completo)
```

```
       pais  pib  inflacao  cambio_medio
0    Brasil  2.2       4.5          5.45
1 Argentina -1.8      98.0           NaN
2     Chile  3.1       NaN        950.20
3      Peru  2.5       NaN           NaN
4  Colômbia  NaN       7.2       4100.00
```

```
# Passo 3: identificar países com dado incompleto (algum NaN na linha)
incompletos = completo[completo.isnull().any(axis=1)]
print(incompletos["pais"].tolist())
# ['Argentina', 'Chile', 'Peru', 'Colômbia']
```

O uso de `how="outer"` em cascata (fonte 1 + fonte 2, depois esse resultado + fonte 3) é o padrão correto quando o objetivo é **não perder nenhum país só porque uma fonte não o cobre** — usar `how="inner"` aqui destruiria silenciosamente três dos cinco países, sobrando só o Brasil (única linha sem nenhum `NaN`).

`df.isnull().any(axis=1)` retorna `True` para qualquer linha que tenha **pelo menos um** `NaN` em qualquer coluna (`axis=1` pede para verificar "ao longo das colunas", isto é, dentro de cada linha) — a forma idiomática de detectar registros incompletos após uma série de merges externos.

Aprofundando a teoria — a explosão de linhas no merge muitos-para-muitos
O `merge` cria uma linha para *cada combinação* de linhas que compartilham a chave. Se a chave se repete nas duas tabelas, o resultado tem $m \times n$ **linhas por chave** — e é um dos erros silenciosos mais graves em análise de dados: totais que multiplicam sem aviso. Antes de qualquer merge, confira a *cardinalidade* das chaves (`df["chave"].is_unique`) e o número de linhas depois.

```
tab_a = pd.DataFrame({"pais": ["BR", "BR", "CL"], "ano": [2024, 2025, 2025], "pib": [3.1,
2.2, 3.1]})
tab_b = pd.DataFrame({"pais": ["BR", "BR", "CL"], "evento": ["eleição", "chuvas",
"plebiscito"]})
print(f"chave única em a? {tab_a['pais'].is_unique} | em b? {tab_b['pais'].is_unique}")
explodido = tab_a.merge(tab_b, on="pais")
print(f"a tem {len(tab_a)} linhas, b tem {len(tab_b)}, merge tem {len(explodido)} (BR
aparece 2 x 2 = 4 vezes):")
print(explodido.to_string(index=False))
```

```
chave única em a? False | em b? False
a tem 3 linhas, b tem 3, merge tem 5 (BR aparece 2 x 2 = 4 vezes):
pais  ano  pib     evento
  BR 2024  3.1    eleição
  BR 2024  3.1     chuvas
  BR 2025  2.2    eleição
  BR 2025  2.2     chuvas
  CL 2025  3.1 plebiscito
```

Exemplo resolvido — `merge_asof`: juntando uma série diária com uma mensal pela data mais recente
Selic (diária) e IPCA 12m (mensal) têm frequências diferentes. O `merge_asof` associa, a cada linha de uma tabela, a observação *mais recente anterior* da outra — sem reamostrar nada:

```
diaria = selic_d.reset_index()
mensal = ipca12.reset_index()
casado = pd.merge_asof(diaria, mensal, on="data", direction="backward")
print(casado.dropna().tail(4).to_string(index=False))
ex_post = (1 + casado["selic_meta"] / 100) / (1 + casado["ipca_12m"] / 100) - 1
print(f"Juro real ex-post (Fisher) no último dia: {ex_post.iloc[-1] * 100:.2f}% a.a. (Selic
{casado['selic_meta'].iloc[-1]}%, IPCA 12m {casado['ipca_12m'].iloc[-1]}%)")
```

```
      data  selic_meta  ipca_12m
2026-09-13        14.0      4.44
2026-09-14        14.0      4.44
2026-09-15        14.0      4.44
2026-09-16        14.0      4.44
Juro real ex-post (Fisher) no último dia: 9.15% a.a. (Selic 14.0%, IPCA 12m 4.44%)
```

Capítulo 11 — Datas e séries temporais

11.1. `to_datetime` e o acessor `.dt`

```
df["data"] = pd.to_datetime(df["data"])         # converte texto/número para datetime64[ns]
df["data"] = pd.to_datetime(df["data"], dayfirst=True)  # trata "01/02/2024" como 1º de
fevereiro

df["ano"] = df["data"].dt.year
df["mes"] = df["data"].dt.month
df["dia"] = df["data"].dt.day
df["trimestre"] = df["data"].dt.quarter
df["dia_semana"] = df["data"].dt.day_name()      # "Monday", "Tuesday", ...
df["fim_de_mes"] = df["data"].dt.is_month_end    # booleano
```

O acessor `.dt` funciona de forma análoga ao `.str` visto no Capítulo 6.3: ele expõe, vetorizado sobre a coluna inteira, um conjunto de propriedades e métodos específicos de datas — só disponível quando a coluna já é `datetime64[ns]` (daí a importância de `parse_dates` na leitura, Capítulo 2.2, ou de `pd.to_datetime` explícito).

11.2. Data como Index — a base de toda série temporal em pandas

```
df["data"] = pd.to_datetime(df["data"])
df = df.set_index("data")

df.loc["2020"]                  # todo o ano de 2020
df.loc["2020-01":"2020-06"]     # primeiro semestre de 2020 (fim INCLUSIVE, Capítulo 4.4)
df.loc["2020-01-15":"2020-03-15"]  # intervalo exato de datas

df[df.index >= "2020-01-01"]     # filtro booleano tradicional, também funciona
```

Quando o Index de um DataFrame é um `DatetimeIndex`, `.loc` ganha um comportamento especial de **parsing parcial de data**: `df.loc["2020"]` entende que você quer o ano inteiro, `df.loc["2020-01"]` entende o mês inteiro — sem que você precise escrever `df.loc["2020-01-01":"2020-01-31"]` manualmente. Essa é uma das razões mais fortes para usar a data como Index em vez de mantê-la como uma coluna comum.

11.3. `resample` — mudando a frequência de uma série temporal

```
df.resample("M").mean()    # agrega para frequência MENSAL, calculando a média de cada mês
df.resample("Q").mean()    # trimestral
df.resample("Y").mean()    # anual
df.resample("D").mean()    # diário
df.resample("W").sum()     # semanal, somando (não fazendo média) os valores de cada semana
```

`resample` vs. `groupby` — qual a diferença conceitual? Os dois seguem o mesmo paradigma split-apply-combine (Capítulo 9.1), mas `groupby` agrupa por **valores discretos de uma coluna** (país, categoria), enquanto `resample` agrupa por **intervalos de tempo fixos** (todo mês, todo trimestre) usando o `DatetimeIndex`. Tecnicamente, `df.resample("M").mean()` é equivalente a agrupar por `(df.index.year, df.index.month)` — mas `resample` também **preenche automaticamente períodos sem nenhuma observação** (um mês sem nenhum dado vira uma linha com `NaN`, em vez de simplesmente não aparecer), o que `groupby` não faz.

> **Nota de versão:** a partir do pandas 2.2, os aliases de frequência de calendário como `"M"` (mês) e `"Y"` (ano) foram renomeados para `"ME"` (*month end*) e `"YE"` (*year end*) para maior clareza, mantendo `"M"`/`"Y"` como sinônimos com aviso de depreciação em versões de transição. Ao escrever código novo, prefira as formas explícitas (`"ME"`, `"QE"`, `"YE"`) se sua versão de pandas já as suportar.

11.4. Defasagem (lag), diferenças e variação percentual

```
df["pib_lag1"] = df["pib"].shift(1)     # valor do período ANTERIOR (defasagem de 1)
df["pib_lag2"] = df["pib"].shift(2)     # defasagem de 2 períodos
df["pib_lead1"] = df["pib"].shift(-1)   # valor do período SEGUINTE ("lead", defasagem
negativa)

df["pib_diff"] = df["pib"].diff()           # diferença absoluta frente ao período anterior
df["pib_pct"] = df["pib"].pct_change() * 100  # variação percentual frente ao período
anterior
```

`pct_change()` calcula, para cada linha, exatamente:

$$\%\Delta x_t = \frac{x_t - x_{t-1}}{x_{t-1}}$$

— a mesma fórmula de taxa de crescimento usada para calcular a variação mensal do IPCA a partir do índice de preços, ou a valorização cambial mês a mês.

11.5. Médias móveis (`rolling`) e janelas expansivas (`expanding`)

```
df["pib_mm3"] = df["pib"].rolling(window=3).mean()    # média móvel de 3 períodos
df["pib_mm12"] = df["pib"].rolling(window=12).mean()  # média móvel de 12 períodos (comum
p/ dados mensais)
df["pib_vol3"] = df["pib"].rolling(window=3).std()    # volatilidade móvel (desvio-padrão
em janela)

# min_periods permite calcular mesmo antes da janela estar "cheia"
df["pib_mm12_parcial"] = df["pib"].rolling(window=12, min_periods=1).mean()

# expanding() usa TODOS os dados até a linha atual (janela que só cresce)
df["pib_media_acumulada"] = df["pib"].expanding().mean()
```

A diferença entre `rolling` e `expanding`: `rolling(window=N)` sempre olha para os últimos `N` pontos (uma "janela deslizante" de tamanho fixo); `expanding()` sempre olha para **todos os pontos desde o início da série** (uma janela que cresce a cada linha) — útil, por exemplo, para calcular a inflação acumulada desde o início do ano corrente, ou uma média histórica que se atualiza a cada novo mês sem "esquecer" o passado.

11.6. Exemplo Resolvido — IPCA acumulado em 12 meses, calculado do zero

Enunciado: o BCB divulga o "IPCA acumulado em 12 meses" como uma série pronta (código SGS 13522), mas é instrutivo saber calculá-lo a partir da série mensal (código 433) — porque a mesma técnica serve para deflacionar qualquer série de valores nominais. Dada a série mensal de IPCA (%), calcule o IPCA acumulado nos últimos 12 meses, mês a mês, usando `rolling`.

```
datas = pd.date_range("2023-01-01", periods=15, freq="MS")  # MS = início do mês
ipca_mensal = pd.Series(
    [0.53, 0.84, 0.71, 0.61, 0.23, 0.08, 0.12, 0.23, 0.26, 0.28, 0.28, 0.56,
     0.42, 0.83, 0.16],
    index=datas,
    name="ipca",
)
```

Solução comentada:

O erro conceitual mais comum aqui é **somar** as 12 taxas mensais. Inflação **composta**, não soma — a mesma distinção entre juros simples e compostos da Apostila 1 (Capítulo 5.1). A fórmula correta para a inflação acumulada em uma janela de n meses é:

$$\text{IPCA}_{\text{acum},t} = \left[\prod_{k=0}^{n-1}\left(1 + \frac{i_{t-k}}{100}\right)\right] - 1$$

```
def acumulado_composto(janela):
    fatores = 1 + janela / 100
    return (fatores.prod() - 1) * 100

ipca_acum_12m = ipca_mensal.rolling(window=12).apply(acumulado_composto)
print(ipca_acum_12m.tail())
```

`rolling(window=12).apply(funcao)` aplica uma função personalizada a cada janela de 12 observações consecutivas — diferente de `rolling(12).mean()`, que já vem pronta, aqui precisamos de uma composição multiplicativa, não uma média, então fornecemos nossa própria função. `janela.prod()` calcula o produtório dos 12 fatores `(1 + i/100)` dentro daquela janela específica — exatamente a mesma lógica usada para compor juros mês a mês (Apostila 1, Capítulo 10.5), só que aplicada automaticamente a cada janela deslizante de 12 meses pela infraestrutura do `rolling`.

Vale registrar: como o `.rolling(window=12)` exige 12 observações completas por padrão, os 11 primeiros meses da série de saída vêm como `NaN` (Capítulo 8) — não há 12 meses anteriores disponíveis para eles ainda.

Exemplo resolvido — `Period`, trimestres, dias úteis, `shift` por frequência e média móvel exponencial
O pandas distingue o *instante* (`Timestamp`) do *período* (`Period`: "março de 2026" inteiro). Períodos facilitam agrupar por mês ou trimestre; `CustomBusinessDay` modela calendários com feriados:

```
datas = pd.Series(pd.to_datetime(["2026-01-15", "2026-04-02", "2026-09-30"]))
print("Período mensal:", datas.dt.to_period("M").astype(str).tolist(), "| trimestre:",
datas.dt.quarter.tolist(),
      "| dia da semana:", datas.dt.day_name().tolist())

feriados = pd.to_datetime(["2026-04-03", "2026-04-21"])                # Sexta-feira
Santa e Tiradentes
dia_util = pd.offsets.CustomBusinessDay(holidays=feriados)
print("Dias úteis de 30/03 a 24/04 de 2026:", len(pd.date_range("2026-03-30", "2026-04-24",
freq=dia_util)),
      "(sem feriados seriam", len(pd.date_range("2026-03-30", "2026-04-24", freq="B")),
")")

serie_m = ipca12.loc["2025-06":"2026-03"]
print("shift(1) traz o valor do período anterior; ewm pesa mais os valores recentes:")
print(pd.DataFrame({"valor": serie_m, "posicao_-1": serie_m.shift(1), "ewm(span=3)":
serie_m.ewm(span=3).mean().round(2)}).head(4).to_string())
```

```
Período mensal: ['2026-01', '2026-04', '2026-09'] | trimestre: [1, 2, 3] | dia da semana:
['Thursday', 'Thursday', 'Wednesday']
Dias úteis de 30/03 a 24/04 de 2026: 18 (sem feriados seriam 20 )
shift(1) traz o valor do período anterior; ewm pesa mais os valores recentes:
            valor  posicao_-1  ewm(span=3)
data
2025-06-01   5.35         NaN         5.35
2025-07-01   5.23        5.35         5.27
2025-08-01   5.13        5.23         5.19
2025-09-01   5.17        5.13         5.18
```

Intuição
A média móvel simples (`rolling`) pesa igualmente as últimas n observações e descarta as anteriores de um golpe; a **exponencial** (`ewm`) dá peso decrescente a todas, sem corte brusco, e reage mais rápido a mudanças recentes. Escolha pelo uso: suavizar uma série (rolling) ou acompanhar uma tendência que muda (ewm).

Capítulo 12 — Estatística descritiva com pandas

12.1. Tendência central e dispersão

```
df["pib"].mean()      # média aritmética
df["pib"].median()    # mediana (mais robusta a outliers que a média)
df["pib"].std()       # desvio-padrão AMOSTRAL (ddof=1 por padrão)
df["pib"].var()       # variância amostral
df["pib"].min()
df["pib"].max()
df["pib"].quantile(0.25)              # 1º quartil
df["pib"].quantile([0.25, 0.5, 0.75])  # vários quantis de uma vez
df["pib"].skew()       # assimetria (skewness)
df["pib"].kurtosis()   # curtose (peso das caudas da distribuição)
```

Média e mediana:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i \qquad \text{mediana} = x_{\left(\frac{n+1}{2}\right)} \ \text{(valor central, dados ordenados)}$$

Armadilha importante: `pandas.std()` e `numpy.std()` usam convenções diferentes por padrão.

$$s_{\text{amostral}} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2} \qquad \sigma_{\text{populacional}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

Chamada	Convenção usada por padrão	Parâmetro para mudar
`df["pib"].std()` (pandas)	Amostral, `ddof=1` (denominador $n-1$)	`ddof=0` para populacional
`np.std(array)` (numpy puro)	Populacional, `ddof=0` (denominador n)	`ddof=1` para amostral

```
serie = pd.Series([1, 2, 3, 4, 5])
print(serie.std())          # 1.5811... (ddof=1, amostral -- padrão do pandas)
print(np.std(serie))        # 1.4142... (ddof=0, populacional -- padrão do numpy!)
print(serie.std(ddof=0))    # 1.4142... (agora bate com numpy)
```

Essa divergência de convenção **entre as duas bibliotecas mais usadas juntas em Python** é uma fonte real de erro sutil: se um economista calcula o desvio-padrão de uma série com `.std()` do pandas em um lugar do código, e com `np.std()` em outro (por exemplo, ao comparar contra um resultado de uma biblioteca de terceiros que só aceita array numpy), os dois números **não vão bater** — não por erro de cálculo, mas por convenção diferente de $n-1$ vs. n. A diferença desaparece à medida que n cresce (a razão $n/(n-1)$ tende a 1), mas é material para amostras pequenas — exatamente o caso comum em séries macroeconômicas anuais, que raramente têm mais que algumas dezenas de observações.

12.2. Correlação

```
df["pib"].corr(df["inflacao"])       # correlação de Pearson entre duas Series
df[["pib", "inflacao", "populacao"]].corr()   # matriz de correlação completa
df.corr(numeric_only=True).round(2)  # arredondada, ignorando colunas de texto
```

A correlação de Pearson entre duas variáveis x e y:

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\,\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}$$

r_{xy} varia entre -1 (correlação negativa perfeita) e $+1$ (correlação positiva perfeita), com 0 indicando ausência de relação **linear** — o que não implica ausência de qualquer relação (duas variáveis podem ter uma relação forte, porém não-linear, e ainda assim $r_{xy} \approx 0$).

Correlação não é causalidade — o alerta mais repetido (e mais frequentemente ignorado) em qualquer curso de estatística ou econometria introdutória. `df.corr()` mede associação estatística, nunca prova que uma variável causa outra: câmbio e inflação podem estar correlacionados porque o câmbio afeta a inflação (*pass-through* cambial), porque a inflação afeta expectativas que afetam o câmbio, ou porque uma terceira variável (por exemplo, a política monetária dos EUA) afeta as duas simultaneamente. A Apostila 4 (Econometria) tratará ferramentas desenhadas para investigar causalidade — regressão com controles, causalidade de Granger, variáveis instrumentais — que exigem hipóteses muito mais fortes do que uma simples correlação.

12.3. Exemplo Resolvido — Comparando volatilidade entre duas décadas

Enunciado: dada uma série de câmbio (R$/US$) diário, compare a volatilidade (desvio-padrão) do período 2010-2014 contra 2020-2024, e verifique se a diferença é grande o suficiente para não ser só ruído estatístico de amostra pequena (usando o raciocínio de `ddof` da Seção 12.1 — sem fazer teste de hipótese formal, que é assunto de estatística inferencial).

```
np.random.seed(42)
datas = pd.date_range("2010-01-01", "2024-12-31", freq="B")  # dias úteis
cambio = pd.Series(
    5.0 + np.cumsum(np.random.normal(0, 0.01, size=len(datas))),
    index=datas,
    name="cambio",
)
```

Solução comentada:

```
periodo_1 = cambio.loc["2010":"2014"]
periodo_2 = cambio.loc["2020":"2024"]

vol_1 = periodo_1.std()
vol_2 = periodo_2.std()

print(f"Volatilidade 2010-2014: {vol_1:.4f}")
print(f"Volatilidade 2020-2024: {vol_2:.4f}")
print(f"n período 1: {len(periodo_1)}, n período 2: {len(periodo_2)}")
```

Repare que ambos os `.std()` usam `ddof=1` (padrão do pandas) automaticamente — o que é o correto aqui, porque estamos tratando cada período como uma **amostra** de um processo cambial mais amplo, não como a população inteira de todos os valores de câmbio possíveis. Se os dois períodos tiverem tamanhos de amostra muito diferentes (n bem diferente), isso por si só já introduz mais incerteza na comparação — outro motivo pelo qual, para uma conclusão estatisticamente rigorosa (não apenas descritiva), seria necessário um teste de hipótese formal (teste F de igualdade de variâncias, por exemplo), fora do escopo desta apostila e reservado para a trilha de econometria.

Aprofundando a teoria — quantis, assimetria, curtose e outliers
A média resume mal distribuições assimétricas ou com caudas gordas; os **quantis** descrevem a forma sem depender de extremos. O **IQR** (intervalo interquartil, $Q_3 - Q_1$) define cercas para outliers: valores fora de $[Q_1 - 1{,}5\ \text{IQR}, Q_3 + 1{,}5\ \text{IQR}]$ merecem inspeção. A **assimetria** (`skew`) mede se a cauda direita (> 0) ou esquerda (< 0) é mais longa; a **curtose** (`kurt`, excesso sobre a normal) mede o peso das caudas.

```
amostra = grupos["Servicos"]
q1, q2, q3 = amostra.quantile([0.25, 0.5, 0.75])
iqr = q3 - q1
cercas = (q1 - 1.5 * iqr, q3 + 1.5 * iqr)
outliers = amostra[(amostra < cercas[0]) | (amostra > cercas[1])]
print(f"Serviços (12m): média {amostra.mean():.2f} | mediana {q2:.2f} | Q1 {q1:.2f} | Q3
{q3:.2f} | IQR {iqr:.2f}")
print(f"Cercas de outlier: ({cercas[0]:.2f}, {cercas[1]:.2f}) | outliers: {len(outliers)} |
assimetria {amostra.skew():.2f} | curtose {amostra.kurt():.2f}")
z = (amostra - amostra.mean()) / amostra.std()
print(f"Maior |z|: {z.abs().max():.2f} (em {z.abs().idxmax():%m/%Y})")
```

```
Serviços (12m): média 5.78 | mediana 5.84 | Q1 5.27 | Q3 6.08 | IQR 0.81
Cercas de outlier: (4.04, 7.30) | outliers: 4 | assimetria 0.88 | curtose 1.13
Maior |z|: 2.56 (em 02/2023)
```

Exemplo resolvido — correlação de Pearson × Spearman e o paradoxo de Simpson
A correlação de Pearson mede associação *linear* a de Spearman, associação *monotônica* (usa postos, é robusta a outliers). E uma correlação calculada sobre a base inteira pode *inverter o sinal* dentro de cada subgrupo — o **paradoxo de Simpson**, armadilha clássica em comparações entre regiões:

```
xs = pd.Series([1, 2, 3, 4, 5, 6, 7, 8, 9, 100])
ys = pd.Series([2, 4, 5, 4, 5, 7, 8, 9, 10, 11])
print(f"Com um outlier -> Pearson {xs.corr(ys):.2f} | Spearman {xs.corr(ys,
method='spearman'):.2f}")

simpson = pd.DataFrame({
    "grupo": ["A"] * 4 + ["B"] * 4,
    "x": [1, 2, 3, 4, 8, 9, 10, 11],
    "y": [9, 8, 7, 6, 20, 19, 18, 17],
})
grupo_a, grupo_b = simpson[simpson["grupo"] == "A"], simpson[simpson["grupo"] == "B"]
print(f"Correlação geral: {simpson['x'].corr(simpson['y']):+.2f} (positiva) | dentro do
grupo A: "
      f"{grupo_a['x'].corr(grupo_a['y']):+.2f} | dentro do B:
{grupo_b['x'].corr(grupo_b['y']):+.2f} (ambas negativas)")
```

```
Com um outlier -> Pearson 0.60 | Spearman 0.97
Correlação geral: +0.87 (positiva) | dentro do grupo A: -1.00 | dentro do B: -1.00 (ambas
negativas)
```

Atenção — erro comum
Correlação **não é causalidade**. Duas séries que crescem no tempo (PIB e número de celulares) têm correlação alta sem nenhuma relação causal — a **correlação espúria**. Séries temporais devem ser comparadas em *variações* (diferenças ou taxas), não em níveis (Apostila 4).

Capítulo 13 — Exportando dados e um script completo do mundo real

13.1. Salvando resultados

```
df.to_csv("dados_processados.csv", index=False)                          # sem a coluna de
índice
df.to_csv("dados_processados.csv", index=False, decimal=",", sep=";")   # formato
"brasileiro"
df.to_excel("dados_processados.xlsx", sheet_name="Dados", index=False)
df.to_parquet("dados_processados.parquet")   # formato binário colunar -- mais rápido e
compacto
                                             # que CSV para bases grandes; preserva dtypes
exatamente
```

`to_parquet` merece menção mesmo sem ter sido usado nos capítulos anteriores: diferente de CSV (que é sempre texto, e por isso perde a informação de `dtype` — tudo volta a ser reinterpretado na leitura), o formato Parquet preserva tipos de dado exatamente, comprime melhor e é lido/escrito mais rápido. Para bases grandes (milhões de linhas, como microdados), é preferível a CSV sempre que o arquivo não precisar ser aberto manualmente no Excel.

13.2. Script completo — o que um economista faz no dia a dia

```
import pandas as pd
import numpy as np
from bcb import sgs

# 1. Baixar dados (BCB SGS: 433 = IPCA mensal, 11 = SELIC diária)
ipca = sgs.get({"IPCA": 433}, start="2015-01-01")
selic = sgs.get({"SELIC": 11}, start="2015-01-01")

# 2. Juntar as duas séries por data
df = pd.merge(ipca, selic, on="Date", how="inner")
df = df.set_index("Date")
df.index = pd.to_datetime(df.index)

# 3. Criar colunas derivadas
df["juro_real_aprox"] = df["SELIC"] - df["IPCA"]
df["ipca_acum_12m"] = df["IPCA"].rolling(window=12).apply(
    lambda janela: ((1 + janela / 100).prod() - 1) * 100
)

# 4. Filtrar o período de interesse (usando o Index datetime, Capítulo 11.2)
recentes = df.loc["2023":]

# 5. Diagnosticar dados faltantes antes de qualquer estatística (Capítulo 8)
print("Nulos por coluna:")
print(recentes.isnull().sum())

# 6. Estatísticas descritivas
print("\nCorrelação IPCA x SELIC (2023+):")
print(recentes[["IPCA", "SELIC"]].corr().round(3))

# 7. Salvar em formato brasileiro
recentes.to_csv("analise_bcb.csv", decimal=",", sep=";")
print("\nArquivo salvo: analise_bcb.csv")
```

Este script resume, em sequência, praticamente todos os capítulos desta apostila: importação de fonte real (Capítulo 2), Index e alinhamento por data (Capítulos 1 e 11), criação vetorizada de colunas (Capítulo 7), diag-

nóstico de nulos antes de estatísticas (Capítulo 8), estatística descritiva (Capítulo 12) e exportação no formato esperado por um leitor brasileiro (Capítulo 13.1). É, deliberadamente, o roteiro que se repete — com variações — em praticamente qualquer análise macroeconômica de curto prazo.

Exemplo resolvido — mini-projeto: lendo a regra de Taylor da Selic com pandas
O repositório guarda a série `taylor_rule_series.csv` gerada pelo script `taylor_rule_selic.py`: a Selic meta, o IPCA 12m, a meta de inflação, a taxa neutra r^*, a Selic *implícita* pela regra de Taylor e a diferença entre elas. O pipeline de um analista — ler, conferir, resumir, salvar — usa quase tudo da apostila:

```
import tempfile, os
tay = pd.read_csv("taylor_rule_series.csv", parse_dates=["data"], index_col="data")
assert tay.index.is_monotonic_increasing and not tay.isna().any().any(), "base
inconsistente"          # validação
print(f"{len(tay)} meses | colunas: {list(tay.columns)}")

tay["selic_acima"] = tay["gap_pp"] > 0                                    # Selic acima
do que a regra sugere?
resumo = tay.groupby(tay.index.year).agg(gap_medio=("gap_pp", "mean"),
pct_meses_acima=("selic_acima", "mean"),
                                         selic_media=("selic", "mean"),
taylor_media=("taylor_selic", "mean"))
resumo["pct_meses_acima"] = (resumo["pct_meses_acima"] * 100).round(0)
print(resumo.round(2).to_string())
print(f"\nCorrelação Selic x Taylor: {tay['selic'].corr(tay['taylor_selic']):.2f} | último
gap: {tay['gap_pp'].iloc[-1]:+.2f} p.p.")

with tempfile.TemporaryDirectory() as pasta:
    caminho = os.path.join(pasta, "resumo_taylor.csv")
    resumo.to_csv(caminho, sep=";", decimal=",")                          # formato
brasileiro
    relido = pd.read_csv(caminho, sep=";", decimal=",", index_col=0)
    print("Releitura idêntica ao original:", np.allclose(relido.values, resumo.values))
```

```
43 meses | colunas: ['selic', 'ipca12m', 'meta', 'r_star', 'taylor_selic', 'gap_pp']
      gap_medio  pct_meses_acima  selic_media  taylor_media
data
2023       3.47            100.0        13.25          9.78
2024       1.24            100.0        10.96          9.72
2025       3.53            100.0        14.56         11.03
2026       4.55            100.0        14.61         10.05

Correlação Selic x Taylor: 0.39 | último gap: +4.09 p.p.
Releitura idêntica ao original: True
```

Intuição
A regra de Taylor diz qual taxa de juros seria "compatível" com a inflação e o hiato do produto; o *gap* positivo indica política monetária mais apertada (juros acima do sugerido). O objetivo aqui não é discutir macroeconomia, e sim mostrar como o pipeline — **ler, validar, agrupar, resumir, exportar** — se aplica a dados reais.

Capítulo 14 — Boas práticas com pandas

Assim como a Apostila 1 fechou a teoria com um capítulo de boas práticas gerais de código, pandas tem hábitos próprios que evitam retrabalho e bugs sutis:

- **Rode `df.info()` logo após qualquer importação.** É o equivalente pandas do "debugging por print" da Apostila 1 — barato, rápido, e revela a maioria dos problemas de tipo e de dado faltante antes que eles se propaguem.
- **Prefira vetorização a `.apply()`, e `.apply()` a um loop `for` manual.** Nessa ordem de preferência (Capítulo 7.3). Reserve `.apply()`/loops para lógica genuinamente impossível de vetorizar.
- **Trate `SettingWithCopyWarning` como um erro, nunca o silencie.** Ele existe porque o pandas não consegue garantir a intenção do seu código (Capítulo 5) — ignorá-lo é assinar um cheque em branco para um bug que vai aparecer mais tarde, em outro lugar do pipeline.
- **Evite `inplace=True`** na maior parte do código novo — prefira reatribuir (`df = df.metodo(...)`), que é mais previsível e permite encadear métodos (*method chaining*):

  ```
  resultado = (
      df.dropna(subset=["pib"])
        .query("pib > 0")
        .assign(pib_milhares=lambda d: d["pib"] * 1000)
        .sort_values("pib", ascending=False)
  )
  ```

  Esse estilo — uma sequência de métodos encadeados, cada um numa linha, formando uma "receita" legível de cima a baixo — é considerado idiomático em pandas moderno, e evita a criação de várias variáveis intermediárias (`df2`, `df3`, `df_final`) que só existem para guardar um passo intermediário.
- **Nomeie colunas em `snake_case`, sem espaços ou acentos quando possível** (`pib_per_capita`, não `"PIB per Capita"`) — evita ter que usar sempre colchetes (nunca o atalho `df.coluna`) e problemas de codificação ao salvar/reabrir arquivos.
- **Sempre confira o `dtype` antes de comparar ou agregar uma coluna.** Uma coluna numérica lida como `object` (Capítulo 3.3/3.4) vai silenciosamente comparar como texto, produzindo resultados sem sentido sem lançar erro algum.

Anti-padrão	Idioma recomendado
`for` sobre linhas com `iterrows`	Operação vetorizada ou `groupby`/`merge`
`apply(lambda ...)` linha a linha em tabelas grandes	Operadores de coluna, `np.where`/`np.select`, `map`
Atribuição encadeada `df[a][b] = x`	`df.loc[a, b] = x`
Colunas de texto repetido como `object`	`astype("category")`
`inplace=True` em toda parte	Reatribuir ou encadear (`df = df.dropna()`)
`concat` dentro de um laço (quadrático)	Coletar numa lista e `pd.concat` uma vez
Conferir resultado "a olho"	`assert`, `pd.testing.assert_frame_equal`, contagem de linhas antes/depois
Datas como texto	`pd.to_datetime(..., format=...)` e `DatetimeIndex`

```
def valida_painel(d: pd.DataFrame) -> pd.DataFrame:
    """Pré-condições de um painel país-ano: falha cedo, com mensagem clara."""
    assert {"pais", "ano", "pib"} <= set(d.columns), "colunas obrigatórias ausentes"
    assert not d.duplicated(["pais", "ano"]).any(), "pares (pais, ano) duplicados"
    assert d["pib"].between(-50, 50).all(), "PIB fora do intervalo plausível"
    return d

bom = pd.DataFrame({"pais": ["BR", "BR", "CL"], "ano": [2024, 2025, 2025], "pib": [3.1,
2.2, 3.1]})
ruim = pd.concat([bom, bom.iloc[[0]]])                                    # duplicata
```

```
acidental
print("painel ok:", len(bom.pipe(valida_painel)), "linhas")
try:
    ruim.pipe(valida_painel)
except AssertionError as erro:
    print("painel rejeitado:", erro)
pd.testing.assert_frame_equal(bom, bom.copy())                                  # compara
valores E tipos
print("assert_frame_equal passou: tabelas idênticas")
```

```
painel ok: 3 linhas
painel rejeitado: pares (pais, ano) duplicados
assert_frame_equal passou: tabelas idênticas
```

Capítulo 15 — Exercícios para Executar (na mão)

Estes exercícios não exigem computador: o objetivo é treinar o raciocínio sobre como pandas se comporta antes de testar no código. Resolva no papel. **As soluções não estão neste documento** — quando terminar, peça para eu conferir suas respostas.

Exercício 1 — Trace de `.loc` vs. `.iloc`
Dado o DataFrame abaixo (índice **não** é 0,1,2,... — foi definido manualmente):

```
df = pd.DataFrame(
    {"pib": [2.2, -1.8, 3.1, 2.8]},
    index=[10, 20, 30, 40],
)
```

Sem rodar no computador, diga o que cada expressão abaixo retorna (quantas linhas, quais valores):

```
a) df.iloc[0:2]
b) df.loc[10:30]
c) df.iloc[1:3]
d) df.loc[20:40]
```

Exercício 2 — Calculando correlação de Pearson na mão
Dadas as séries de PIB e inflação de 4 anos:

```
pib:      [1.0, 2.0, 3.0, 4.0]
inflacao: [8.0, 6.0, 4.0, 2.0]
```

Calcule, usando papel e caneta (ou calculadora, mas sem `pandas`/`numpy`), a correlação de Pearson r_{xy} entre as duas séries, usando a fórmula da Seção 12.2. Que sinal (positivo ou negativo) você esperava antes de calcular, só olhando o padrão das duas séries?

Exercício 3 — Encontre o erro: `SettingWithCopyWarning`
O trecho abaixo gera um `SettingWithCopyWarning`. Explique, em suas próprias palavras, por que o aviso ocorre (o que pandas não consegue garantir) e reescreva o código de duas formas diferentes: uma que garanta que `df` original seja modificado, e outra que garanta que `df` original **não** seja modificado.

```
df = pd.DataFrame({"pais": ["Brasil", "Chile"], "pib": [2.2, 3.1]})
alta = df[df["pib"] > 2.5]
alta["destaque"] = True
```

Exercício 4 — Pseudocódigo do split-apply-combine
Descreva, em português estruturado (sem precisar ser código Python válido, no mesmo espírito do Exercício 5 da Apostila 1), o algoritmo que `df.groupby("regiao")["pib"].mean()` executa internamente, nomeando explicitamente as três etapas (split, apply, combine) e o que acontece em cada uma.

Exercício 5 — Prevendo dtypes após um merge externo
Duas tabelas são unidas com `how="outer"`: a primeira tem uma coluna `populacao` como `int64`, sem nenhum valor faltante; a segunda tabela não tem essa coluna. Depois do merge, algumas linhas (as que só existiam na segunda tabela) não têm valor de `populacao`. Sem rodar no computador, diga qual será o `dtype` final da coluna `populacao` no resultado, e explique o porquê usando o conceito do Capítulo 8.1.

Exercício 6 — `rolling` vs. `expanding`, na mão
Dada a série de PIB trimestral `[1.0, 2.0, -1.0, 3.0, 0.5]` (índices 0 a 4), calcule manualmente:

a) `rolling(window=2).mean()` para cada posição (mostre `NaN` onde não houver 2 observações completas).
b) `expanding().mean()` para cada posição.

Compare os dois resultados na posição de índice 4 e explique a diferença conceitual entre as duas janelas.

Capítulo 16 — Exercícios para Executar (em código)

Implemente e execute cada um no seu editor ou notebook. **As soluções não estão neste documento** — o objetivo é você rodar de verdade e ver o resultado; quando terminar, peça para eu revisar seu código.

Exercício 1 — Primeira exploração
Crie um DataFrame com PIB, inflação, desemprego e câmbio médio de 6 países de sua escolha. Rode `df.info()`, `df.describe()` e `df.dtypes`, e escreva um comentário para cada coluna dizendo se o `dtype` está como esperado.

Exercício 2 — `.loc` vs `.iloc` na prática
Usando o DataFrame do Exercício 1 com o índice trocado para as siglas dos países (`df.index = [...]`), escreva quatro seleções diferentes: uma linha por posição, uma linha por rótulo, um intervalo de linhas por posição, e um intervalo de linhas por rótulo. Confirme se o resultado bate com o que você esperava sobre inclusão/exclusão do limite final.

Exercício 3 — Reproduzindo o `SettingWithCopyWarning`
Escreva deliberadamente um código que produza o aviso (indexação encadeada), confirme que o aviso aparece, e então reescreva-o de forma segura usando `.loc` em uma única chamada.

Exercício 4 — Filtro composto
Dado o DataFrame abaixo, filtre os países com PIB positivo **e** inflação abaixo de 10%, depois filtre os países com PIB negativo **ou** inflação acima de 50%. Refaça o segundo filtro usando `.query()`.

```
df = pd.DataFrame({
    "pais": ["Brasil", "Argentina", "Chile", "Turquia", "Peru"],
    "pib": [2.2, -1.8, 3.1, 3.0, 2.5],
    "inflacao": [4.5, 98.0, 3.8, 55.0, 3.2],
})
```

Exercício 5 — `np.select` substituindo `.apply()`
Reescreva a classificação abaixo (atualmente feita com `.apply()`) usando `np.select`, e compare os dois resultados para garantir que são idênticos:

```
def classifica(pib):
    if pib > 3:
        return "Alto"
```

```
    elif pib > 0:
        return "Médio"
    else:
        return "Negativo"

df["categoria_apply"] = df["pib"].apply(classifica)
```

Exercício 6 — Diagnóstico e tratamento de nulos
Dado o DataFrame abaixo, rode `isnull().sum()`, decida (e justifique em comentário) uma estratégia de tratamento diferente para cada coluna com base na Seção 8.4, e aplique-a.

```
df = pd.DataFrame({
    "data": pd.date_range("2024-01-01", periods=10, freq="D"),
    "cambio": [5.10, 5.12, None, None, 5.15, 5.14, 5.16, None, 5.18, 5.20],
    "ipca_mensal": [0.42, None, None, None, None, None, None, None, None, None],
})
```

Exercício 7 — `groupby` com múltiplas agregações nomeadas
Usando o DataFrame de PIB por país e ano (crie um com pelo menos 3 países e 4 anos), calcule, por país: média, desvio-padrão, mínimo e máximo do PIB, usando agregação nomeada (`.agg(nome=("coluna", "função"))`).

Exercício 8 — `transform` para desvio em relação ao grupo
Usando o mesmo DataFrame do Exercício 7, crie uma coluna com o desvio de cada observação de PIB em relação à média do seu próprio país, usando `.groupby(...).transform("mean")`.

Exercício 9 — Merge com `indicator` e auditoria
Crie duas tabelas de países parcialmente sobrepostas (cada uma com países que a outra não tem). Faça um merge `outer` com `indicator=True` e conte quantas linhas vieram de `"left_only"`, `"right_only"` e `"both"`.

Exercício 10 — `validate` capturando um merge indevido
Crie duas tabelas em que a coluna-chave tenha uma duplicata proposital em uma delas. Rode o merge sem `validate` e observe o número de linhas resultante; depois rode com `validate="one_to_one"` e confirme que um erro é lançado.

Exercício 11 — `concat` de múltiplos anos
Crie três DataFrames pequenos, um para cada ano (2022, 2023, 2024), todos com as mesmas colunas. Empilhe os três com `pd.concat`, usando `keys=` para conseguir depois identificar de qual ano cada linha veio, mesmo após juntar tudo.

Exercício 12 — Índice de tempo e fatiamento por período
Crie uma série diária de 3 anos de um valor simulado de câmbio (pode usar `np.random`), com o Index como `DatetimeIndex`. Filtre: (a) o ano do meio inteiro, (b) o segundo trimestre do primeiro ano, (c) um intervalo arbitrário de 10 dias.

Exercício 13 — `resample` mensal a partir de dados diários
Usando a série do Exercício 12, calcule a média mensal (`resample("M")`) e a média trimestral. Compare o número de linhas resultante em cada caso.

Exercício 14 — Defasagem, diferença e variação percentual
Usando uma série mensal de 24 meses de um indicador de sua escolha, crie colunas com defasagem de 1 e 3 períodos, diferença absoluta, e variação percentual. Verifique manualmente (para as duas primeiras linhas) se os valores calculados por pandas batem com o que você esperaria.

Exercício 15 — Reconstruindo o IPCA acumulado em 12 meses
Reproduza o Exemplo Resolvido do Capítulo 11.6 com uma série de 30 meses de sua escolha (valores entre −0.5% e 1.5%, por exemplo). Depois, confira o resultado do último mês somando manualmente (sem compor) as 12

taxas — mostre que a soma simples e o resultado composto são diferentes, e explique por que o composto é o correto.

Exercício 16 — `std` do pandas vs. `numpy`, lado a lado
Crie uma Series com 5 valores. Calcule o desvio-padrão de quatro formas: `.std()` do pandas (padrão), `.std(ddof=0)`, `np.std()` (padrão) e `np.std(ddof=1)`. Confirme quais pares batem entre si e explique o porquê usando o conceito da Seção 12.1.

Exercício 17 — Matriz de correlação com `numeric_only`
Crie um DataFrame com pelo menos 4 colunas numéricas e 1 coluna de texto (nome do país). Calcule a matriz de correlação completa, arredondada a 2 casas decimais, sem que a coluna de texto cause erro.

Exercício 18 — Método encadeado (*method chaining*)
Reescreva o pipeline abaixo (escrito com variáveis intermediárias) como uma única cadeia de métodos, no estilo do Capítulo 14:

```
df1 = df.dropna(subset=["pib"])
df2 = df1[df1["pib"] > 0]
df3 = df2.sort_values("pib", ascending=False)
df3["pib_milhares"] = df3["pib"] * 1000
resultado = df3
```

Exercício 19 — Exportando em dois formatos
Dado qualquer DataFrame de sua criação com pelo menos 3 colunas numéricas, exporte-o em `.csv` (formato brasileiro: `;` e `,`) e em `.xlsx`. Releia os dois arquivos de volta em DataFrames separados e confirme, com `.dtypes`, se algum tipo de dado mudou entre a exportação e a releitura.

Exercício 20 — `category` e uso de memória
Crie uma coluna de texto com 100.000 linhas, mas apenas 5 valores distintos repetidos (por exemplo, região do país). Compare `df.memory_usage(deep=True)` antes e depois de converter essa coluna para `dtype="category"`, e reporte a economia de memória em percentual.

Exercício 21 — Projeto integrador: painel BCB completo
Combine praticamente tudo do capítulo: baixe (ou simule, se não tiver acesso à internet, usando `np.random` com uma tendência) o IPCA mensal e a SELIC diária de 2018 até hoje; junte as duas séries por mês (reamostrando a SELIC diária para mensal antes); calcule juro real aproximado, IPCA acumulado em 12 meses e uma média móvel de 3 meses do juro real; filtre para os últimos 3 anos; calcule a matriz de correlação; e salve o resultado final em CSV no formato brasileiro. Estruture o script em passos comentados, como no Capítulo 13.2.

Capítulo Final — Resumo

Operação	Código
Abrir CSV (com opções brasileiras)	`pd.read_csv("f.csv", sep=";", decimal=",", parse_dates=["data"])`
Ver estrutura/tipos	`df.info()` / `df.dtypes`
Selecionar por posição	`df.iloc[0:3]` (fim exclusivo)
Selecionar por rótulo	`df.loc["a":"c"]` (fim inclusivo!)
Filtrar linhas	`df[(df["x"] > 0) & (df["y"] < 10)]`
Cópia explícita e segura	`sub = df[mascara].copy()`
Atribuir com filtro, sem aviso	`df.loc[mascara, "col"] = valor`
Nova coluna vetorizada	`df["nova"] = df["x"] * 2`

Operação	Código
Classificação vetorizada	`np.select([...], [...], default=...)`
Checar/tratar nulos	`df.isnull().sum()` / `df.fillna(...)` / `df.dropna(...)`
Agrupar	`df.groupby("x")["y"].agg(media=("y","mean"))`
Desvio em relação ao grupo	`df.groupby("x")["y"].transform("mean")`
Juntar tabelas por chave	`pd.merge(df1, df2, on="x", how="outer")`
Empilhar tabelas	`pd.concat([df1, df2], ignore_index=True)`
Data como índice	`df.set_index("data")`
Filtrar por período	`df.loc["2020-01":"2020-06"]`
Mudar frequência	`df.resample("M").mean()`
Lag / diferença / variação %	`df["x"].shift(1)` / `.diff()` / `.pct_change()`
Média móvel	`df["x"].rolling(3).mean()`
Desvio-padrão amostral	`df["x"].std()` (pandas: ddof=1; numpy: ddof=0!)
Correlação	`df.corr(numeric_only=True)`
Salvar	`df.to_csv("saida.csv", decimal=",", sep=";")`

Capítulo 17 — Erros Comuns e Teste Rápido

17.1. Os erros que mais aparecem

#	Erro	Como evitar
1	Usar `and`/`or` em máscaras	&, |, ~ com parênteses
2	`.loc[a:b]` esperando fim exclusivo	`.loc` inclui os dois extremos; `.iloc` exclui o final
3	Atribuição encadeada `df[mascara]["x"] = 0`	`df.loc[mascara, "x"] = 0`
4	Comparar com `np.nan` usando ==	`isna()` / `notna()`
5	Merge sem checar a chave	`is_unique`, `validate=`, contar linhas antes e depois
6	Datas como texto, ordenação lexicográfica	`to_datetime` e `DatetimeIndex`
7	Comparar séries em nível	Compare variações (`pct_change`, `diff`)
8	`std()` do pandas × `np.std()`	pandas usa `ddof=1`; NumPy, `ddof=0`
9	Preencher dado faltante sem entender a causa	Diagnosticar MCAR/MAR/MNAR e a natureza da série
10	Resultado "parece certo" sem teste	Asserções e conferência de totais

17.2. Teste rápido: verdadeiro ou falso (com gabarito)

1. **V ou F:** `df.loc["a":"c"]` inclui a linha "c". **Verdadeiro.** `.loc` é inclusivo nos dois extremos.
2. **V ou F:** Uma coluna de inteiros com um `NaN` continua `int64`. **Falso.** Vira `float64` (ou use `Int64`).
3. **V ou F:** `merge` com chave duplicada nas duas tabelas devolve o mesmo número de linhas. **Falso.** Multiplica as linhas.
4. **V ou F:** `groupby().size()` e `.count()` dão o mesmo resultado com `NaN`. **Falso.** `size` conta linhas; `count` ignora `NaN`.
5. **V ou F:** `ffill` é adequado para a meta da Selic. **Verdadeiro.** É uma série em degraus.
6. **V ou F:** Correlação alta entre duas séries em nível prova causalidade. **Falso.** Pode ser espúria.

7. **V ou F:** `category` reduz memória em colunas com poucos valores distintos repetidos. **Verdadeiro.**
8. **V ou F:** `rolling(3).mean()` e `ewm(span=3).mean()` dão o mesmo resultado. **Falso.** A segunda pesa as observações de forma decrescente.
9. **V ou F:** No pandas 3.0, `df[m]["x"] = 0` altera o `df` original. **Falso.** Com o Copy-on-Write, não altera.
10. **V ou F:** `pd.cut` classifica valores em faixas definidas por você. **Verdadeiro.**

Próxima apostila

Quando terminar os exercícios acima, siga para a **Apostila 3 — Visualização de Dados (matplotlib e seaborn)**. Lá você vai aprender a transformar as tabelas que já sabe manipular em gráficos prontos para publicação — séries temporais, dispersões, histogramas e painéis com múltiplos subplots — usando as mesmas fontes de dados brasileiras (BCB, Ipeadata, IBGE) e os mesmos DataFrames construídos nesta apostila.